

# Basics of Object Oriented Programming (OOP)

---

## Object-oriented programming (OOP):

- The basic unit of OOP is a class, which encapsulates both the static properties and dynamic operations within a "box" and specifies the public interface for using these boxes. Since classes are well encapsulated, it is easier to reuse these classes. In other words, OOP combines the data structures and algorithms of a software entity inside the same box.
- OOP languages permit a higher level of abstraction for solving real-life problems. The traditional procedural language (such as C and Pascal) forces you to think in terms of the structure of the computer (e.g., memory bits and bytes, array, decision, loop) rather than thinking in terms of the problem you are trying to solve. The OOP languages (such as Java, C++ and C#) let you think in the problem space and use software objects to represent and abstract entities of the problem space to solve the problem.

As an example, suppose you wish to write a computer soccer game (which I consider a complex application). It is quite difficult to model the game in procedural-oriented languages. But using OOP languages, you can easily model the program according to the "real things" that appear in the soccer games.

- Player: attributes include name, number, x, and y location in the field, etc; operations include run, jump, kick-the-ball, etc.
- Ball: attributes include x, y, z position in the field, radius, weight, etc.
- Referee:
- Field:
- Audience:
- Weather:

Most importantly, some of these classes (such as `Ball` and `Audience`) can be reused in another application, e.g., computer basketball game, with little or no modification.

## Benefits of OOP

The procedural-oriented languages focus on procedures, with function as the basic unit. You need to first figure out all the functions and then think about how to represent data.

The object-oriented languages focus on components that the user perceives, with objects as the basic unit. You figure out all the objects by putting all the data and operations that describe the user's interaction with the data.

Object-Oriented technology has many benefits:

- Ease in software design as you could think in the problem space rather than the machine's bits and bytes. You are dealing with high-level concepts and abstractions. Ease in design leads to more productive software development.
- Ease in software maintenance: object-oriented software are easier to understand, therefore easier to test, debug, and maintain.

- Reusable software: you don't need to keep re-inventing the wheels and re-write the same functions for different situations. The fastest and safest way of developing a new application is to reuse existing codes - fully tested and proven codes.

## Class & Instances

In Java, a class is a definition of objects of the same kind. In other words, a class is a blueprint, template, or prototype that defines and describes the static attributes and dynamic behaviors common to all objects of the same kind.

An instance is a realization of a particular item of a class. In other words, an instance is an instantiation of a class. All the instances of a class have similar properties, as described in the class definition. For example, you can define a class called "Student" and create three instances of the class "Student" for "Alice", "Ah Beng" and "Ali".

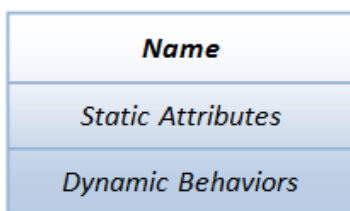
The term "object" usually refers to instance. But it is often used loosely and may refer to a class or an instance.

A Class is a 3-compartment Box Encapsulating Data and Operations

A class can be visualized as a three-compartment box, as illustrated:

- Name (or identity): identifies the class.
- Variables (or attribute, state, field): contains the static attributes of the class.
- Methods (or behaviors, function, operation): contains the dynamic behaviors of the class.

In other words, a class encapsulates the static attributes (data) and dynamic behaviors (operations that operate on the data) in a box.



A class is a 3-compartment box

The following figure shows a few examples of classes:

|                                         | Student               | Circle                   | SoccerPlayer                             | Car                                            |
|-----------------------------------------|-----------------------|--------------------------|------------------------------------------|------------------------------------------------|
| <b>Name</b><br>(Identifier)             |                       |                          |                                          |                                                |
| <b>Variables</b><br>(Static attributes) | name<br>gpa           | radius<br>color          | name<br>number<br>xLocation<br>yLocation | plateNumber<br>xLocation<br>yLocation<br>speed |
| <b>Methods</b><br>(Dynamic behaviors)   | getName()<br>setGpa() | getRadius()<br>getArea() | run()<br>jump()<br>kickBall()            | move()<br>park()<br>accelerate()               |

**Examples of classes**

The following figure shows two instances of the class `Student`, identified as "paul" and "peter".

|                  |                            |                             |
|------------------|----------------------------|-----------------------------|
| <b>Name</b>      | <u>paul:Student</u>        | <u>peter:Student</u>        |
| <b>Variables</b> | name="Paul Lee"<br>gpa=3.5 | name="Peter Tan"<br>gpa=3.9 |
| <b>Methods</b>   | getName()<br>setGpa()      | getName()<br>setGpa()       |

**Two instances - paul and peter - of the class Student**

### Brief Summary

- A **class** is a programmer-defined, abstract, self-contained, reusable software entity that mimics a real-world thing.
- A class is a 3-compartment box containing the name, variables, and methods.
- A class encapsulates the data structures (in variables) and algorithms (in methods). The values of the variables constitute its state. The methods constitute its behaviors.
- An **instance** is an instantiation (or realization) of a particular item of a class.

## More on Object Oriented Programming - Defining classes

---

In Java, we use the keyword `class` to define a class. For examples:

```
public class Circle {           // class name
    double radius;              // variables
    String color;

    double getRadius() { ..... } // methods
    double getArea() { ..... }

}
```

The syntax for class definition in Java is:

```
[AccessControlModifier] class ClassName {
    // Class body contains members (variables and methods)
    .....

}
```

### Creating Instances of a Class

To create an instance of a class, you have to:

- Declare an instance identifier (instance name) of a particular class.
- Construct the instance (i.e., allocate storage for the instance and initialize the instance) using the "new" operator.

For examples, suppose that we have a class called `Circle`, we can create instances of `Circle` as follows:

```
// Declare 3 instances of the class Circle, c1, c2, and c3
Circle c1, c2, c3; // They hold a special value called null
// Construct the instances via new operator
c1 = new Circle();
c2 = new Circle(2.0);
c3 = new Circle(3.0, "red");

// You can Declare and Construct in the same statement
Circle c4 = new Circle();
```

When an instance is declared but not constructed, it holds a special value called `null`.

## Dot (.) Operator

The variables and methods belonging to a class are formally called member variables and member methods. To reference a member variable or method, you must:

- First identify the instance you are interested in, and then,
- Use the dot operator (.) to reference the desired member variable or method.

For example, suppose that we have a class called `Circle`, with two member variables (`radius` and `color`) and two member methods (`getRadius()` and `getArea()`). We have created three instances of the class `Circle`, namely, `c1`, `c2` and `c3`. To invoke the method `getArea()`, you must first identify the instance of interest, say `c2`, then use the dot operator, in the form of `c2.getArea()`.

For example,

```
// Suppose that the class Circle has variables radius and color,
// and methods getArea() and getRadius().
// Declare and construct instances c1 and c2 of the class Circle
Circle c1 = new Circle ();
Circle c2 = new Circle ();
// Invoke member methods for the instance c1 via dot operator
System.out.println(c1.getArea());
System.out.println(c1.getRadius());
// Reference member variables for instance c2 via dot operator
c2.radius = 5.0;
```

```
c2.color = "blue";
```

Calling `getArea()` without identifying the instance is meaningless, as the radius is unknown (there could be many instances of `Circle` - each maintaining its own radius). Furthermore, `c1.getArea()` and `c2.getArea()` are likely to produce different results.

In general, suppose there is a class called `AClass` with a member variable called `aVariable` and a member method called `aMethod()`. An instance called `anInstance` is constructed for `AClass`. You use `anInstance.aVariable` and `anInstance.aMethod()`.

## Member Variables

A member variable has a name (or identifier) and a type and holds a value of that particular type (as described in the earlier chapter).

**Variable Naming Convention:** A variable name shall be a noun or a noun phrase comprising several words. The first word is in lowercase and the rest of the words are initial-capitalized (camel-case), e.g., `fontSize`, `roomNumber`, `xMax`, `yMin` and `xTopLeft`.

The formal syntax for variable definition in Java is:

```
[AccessControlModifier] type variableName [= initialValue];  
[AccessControlModifier] type variableName-1 [= initialValue-1] [, type  
variableName-2 [= initialValue-2]] ... ;
```

For example,

```
private double radius;  
  
public int length = 1, width = 1;
```

## Member Methods

A method (as described in the earlier chapter):

- receives arguments from the caller,
- performs the operations defined in the method body, and
- returns a piece of result (or `void`) to the caller.

The syntax for method declaration in Java is as follows:

```
[AccessControlModifier] returnType methodName ([parameterList]) {  
    // method body or implementation  
    .....  
}
```

For examples:

```
// Return the area of this Circle instance  
public double getArea() {  
    return radius * radius * Math.PI;  
}
```

**Method Naming Convention:** A method name shall be a verb, or a verb phrase made up of several words. The first word is in lowercase and the rest of the words are initial-capitalized (camel-case). For example, `getArea()`, `setRadius()`, `getParameterValues()`, `hasNext()`.

Variable name vs. Method name vs. Class name: A variable name is a noun, denoting an attribute; while a method name is a verb, denoting an action. They have the same naming convention (the first word in lowercase and the rest are initial-capitalized). Nevertheless, you can easily distinguish them from the context. Methods take arguments in parentheses (possibly zero arguments with empty parentheses), but variables do not. In this writing, methods are denoted with a pair of parentheses, e.g., `println()`, `getArea()` for clarity.

On the other hand, class name is a noun beginning with uppercase.

## Writing an OOP Example

---

### Class Definition

| <b>Circle</b>                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------|
| -radius:double=1.0<br>-color:String="red"                                                                                      |
| +Circle()<br>+Circle(r:double)<br>+Circle(r:double,c:String)<br>+getRadius():double<br>+getColor():String<br>+getArea():double |

### Instances

| <b><u>c1:Circle</u></b>                   | <b><u>c2:Circle</u></b>                   | <b><u>c3:Circle</u></b>                   |
|-------------------------------------------|-------------------------------------------|-------------------------------------------|
| -radius=2.0<br>-color="blue"              | -radius=2.0<br>-color="red"               | -radius=1.0<br>-color="red"               |
| +getRadius()<br>+getColor()<br>+getArea() | +getRadius()<br>+getColor()<br>+getArea() | +getRadius()<br>+getColor()<br>+getArea() |

A class called `Circle` is defined as shown in the class diagram. It contains two private member variables: `radius` (of type `double`) and `color` (of type `String`); and three public member methods: `getRadius()`, `getColor()`, and `getArea()`.

Three instances of `Circles`, called `c1`, `c2`, and `c3`, shall be constructed with their respective data members, as shown in the instance diagrams.

The source codes for `Circle.java` is as follows:

### **Circle.java**

```

1  /**
2   * The Circle class models a circle with a radius and color.
3   */
4  public class Circle {    // Save as "Circle.java"
5      // Private instance variables
6      private double radius;
7      private String color;
8
9      // Constructors (overloaded)
10     /** Constructs a Circle instance with default radius and color */
11     public Circle() {          // 1st Constructor (default
12 constructor)
13         radius = 1.0;
14         color = "red";
15     }
16     /** Constructs a Circle instance with the given radius and default
17 color*/
18     public Circle(double r) {    // 2nd Constructor
19         radius = r;
20         color = "red";
21     }
22     /** Constructs a Circle instance with the given radius and color */
23     public Circle(double r, String c) { // 3rd Constructor
24         radius = r;
25         color = c;
26     }
27
28     // Public methods
29     /** Returns the radius */
30     public double getRadius() { // getter for radius
31         return radius;
32     }
33     /** Returns the color */
34     public String getColor() { // getter for color
35         return color;
36     }
37     /** Returns the area of this circle */
38     public double getArea() {
39         return radius * radius * Math.PI;
40     }
41 }

```

**Compile "Circle.java" into "Circle.class"**

Notice that the `Circle` class does not have a `main()` method. Hence, it is NOT a standalone program and you cannot run the `Circle` class by itself. The `Circle` class is meant to be a building block - to be used in other programs.

### **TestCircle.java**

We shall now write another class called `TestCircle`, which uses the `Circle` class. The `TestCircle` class has a `main()` method and can be executed.

```
1  /**
2   * A Test Driver for the "Circle" class
3   */
4  public class TestCircle {    // Save as "TestCircle.java"
5      public static void main(String[] args) {    // Program entry point
6          // Declare and Construct an instance of the Circle class called c1
7          Circle c1 = new Circle(2.0, "blue");    // Use 3rd constructor
8          System.out.println("The radius is: " + c1.getRadius());    // use
9  dot operator to invoke member methods
10         //The radius is: 2.0
11         System.out.println("The color is: " + c1.getColor());
12         //The color is: blue
13         System.out.printf("The area is: %.2f\n", c1.getArea());
14         //The area is: 12.57
15
16         // Declare and Construct another instance of the Circle class
17  called c2
18         Circle c2 = new Circle(2.0);    // Use 2nd constructor
19         System.out.println("The radius is: " + c2.getRadius());
20         //The radius is: 2.0
21         System.out.println("The color is: " + c2.getColor());
22         //The color is: red
23         System.out.printf("The area is: %.2f\n", c2.getArea());
24         //The area is: 12.57
25
26         // Declare and Construct yet another instance of the Circle class
27  called c3
28         Circle c3 = new Circle();    // Use 1st constructor
29         System.out.println("The radius is: " + c3.getRadius());
30         //The radius is: 1.0
31         System.out.println("The color is: " + c3.getColor());
32         //The color is: red
33         System.out.printf("The area is: %.2f\n", c3.getArea());
34         //The area is: 3.14
35     }
36 }
```

**Compile** `TestCircle.java` into `TestCircle.class`.

```
javac TestCircle.java
```



Run the `TestCircle` and study the outputs (shown in blue).

```
java TestCircle
```

## Constructors

A constructor is a special method with the same method name as the class name. That is, the constructor of the class `Circle` is called `Circle()`. In the above `Circle` class, we define three overloaded versions of the constructor `Circle(...)`. A constructor is used to construct and initialize all the member variables. To construct a new instance of a class, you need to use a special "new" operator followed by a call to one of the constructors. For example,

```
Circle c1 = new Circle();           // use 1st constructor
Circle c2 = new Circle(2.0);        // use 2nd constructor
```

```
Circle c3 = new Circle(3.0, "red"); // use 3rd constructor
```

A constructor method is different from an ordinary method in the following aspects:

- The name of the constructor method must be the same as the class name. By class name's convention, it begins with an uppercase (instead of lowercase for ordinary methods).
- The constructor has no return type in its method heading. It implicitly returns `void`. No `return` statement is allowed inside the constructor's body.
- A constructor can only be invoked via the "new" operator. It can only be used once to initialize the instance constructed. Once an instance is constructed, you cannot call the constructor anymore.
- Constructors are not inherited (to be explained later). Every class shall define its own constructors.

Default Constructor: A constructor with no parameter is called the default constructor. It initializes the member variables to their default values. For example, the `Circle()` in the above example, initialize member variables `radius` and `color` to their default values.

## Method Overloading

Method overloading means the same method name can have different implementations (versions). However, the different implementations must be distinguishable by their parameter list (either the number of parameters, or the type of parameters, or their order).

Example: The method `average()` has 3 versions with different parameter lists. The caller can invoke the chosen version by supplying the matching arguments.

```

1  /**
2   * Example to illustrate "Method Overloading"
3   */
4  public class MethodOverloadingTest {
5      public static int average(int n1, int n2) {           // version 1
6          System.out.println("Run version 1");
7          return (n1+n2)/2;
8      }
9      public static double average(double n1, double n2) { // version 2
10         System.out.println("Run version 2");
11         return (n1+n2)/2;
12     }
13     public static int average(int n1, int n2, int n3) {   // version 3
14         System.out.println("Run version 3");
15         return (n1+n2+n3)/3;
16     }
17
18     public static void main(String[] args) {
19         System.out.println(average(1, 2));
20         //Run version 1
21         //1
22         System.out.println(average(1.0, 2.0));
23         //Run version 2
24         //1.5
25         System.out.println(average(1, 2, 3));
26         //Run version 3
27         //2
28         System.out.println(average(1.0, 2));
29         //Run version 2 (int 2 implicitly casted to double 2.0)
30         //1.5
31
32         //average(1, 2, 3, 4);
33         //compilation error: no suitable method found for
34         average(int,int,int,int)
35     }
36 }

```

## Overloading Circle Class' Constructor

Constructor, like an ordinary method, can also be overloaded. The above `Circle` class has three overloaded versions of constructors differentiated by their parameter list, as follows:

```

Circle() // the default constructor
Circle(double r)

Circle(double r, String c)

```

Depending on the actual argument list used when invoking the method, the matching constructor will be invoked. If your argument list does not match any one of the methods, you will get a compilation error.

You can use the keyword `this` to refer to this instance inside a class definition. One of the main uses of keywords `this` is to resolve ambiguity.

```
public class Circle {
    double radius;                // member variable called "radius"
    public Circle(double radius) { // method's parameter also called "radius"
        this.radius = radius;
        // "radius = radius" does not make sense!
        // "this.radius" refers to this instance's member variable
        // "radius" resolved to the method's parameter.
    }
    ...
}
```

In the above codes, there are two identifiers called `radius` - a member variable of the class and the method's parameter. This causes naming conflict. To avoid the naming conflict, you could name the method's argument `r` instead of `radius`. However, `radius` is more approximate and meaningful in this context. Java provides a keyword called `this` to resolve this naming conflict. `"this.radius"` refers to the member variable; while `"radius"` resolves to the method's argument.

Using the keyword `"this"`, the constructor, getter and setter methods for a private variable called `xxx` of type `T` are as follows:

```
public class Ccc {
    // A private variable named xxx of the type T
    private T xxx;

    // Constructor
    public Ccc(T xxx) {
        this.xxx = xxx;
    }

    // A getter for variable xxx of type T receives no argument and returns a
    // value of type T
    public T getXxx() {
        return xxx; // or "return this.xxx" for clarity
    }

    // A setter for variable xxx of type T receives a parameter of type T and
    // return void
    public void setXxx(T xxx) {
        this.xxx = xxx;
    }
}
```

```
}
```

For a boolean variable `xxx`, the getter shall be named `isXxx()` or `hasXxx()`, which is more meaningful than `getXxx()`. The setter remains as `setXxx()`.

```
// private boolean variable
private boolean xxx;

// getter
public boolean isXxx() {
    return xxx; // or "return this.xxx" for clarity
}

// setter
public void setXxx(boolean xxx) {
    this.xxx = xxx;
}

}
```

### More on "this"

- `this.varName` refers to `varName` of this instance; `this.methodName(...)` invokes `methodName(...)` of this instance.
- In a constructor, we can use `this(...)` to call another constructor of this class.
- Inside a method, we can use the statement `"return this"` to return this instance to the caller.

Every well-designed Java class shall have a public method called `toString()` that returns a string description of this instance. You can invoke the `toString()` method explicitly by calling `anInstanceName.toString()`, or implicitly via `println()` or String concatenation operator `'+'`. That is, running `println(anInstance)` invokes the `toString()` method of that instance implicitly.

For example, include the following `toString()` method in our `Circle` class:

```
// Return a String description of this instance
public String toString() {
    return "Circle[radius=" + radius + ",color=" + color + "]";
}

}
```

In your `TestCircle` class, you can get a description of a `Circle` instance via:

```
Circle c4 = new Circle();
System.out.println(c4.toString()); // Explicitly calling toString()
//Circle[radius=1.0,color=red]
System.out.println(c4);           // Implicit call to c4.toString()
//Circle[radius=1.0,color=red]
```

```
System.out.println("c4 is: " + c4);    // '+' invokes toString() to get a
String before concatenation
```

```
//Circle[radius=1.0,color=red]
```

The signature of `toString()` is:

```
public String toString() { ..... }
```

## Constants (final)

Constants are variables defined with the modifier `final`. A `final` variable can only be assigned once and its value cannot be modified once assigned. For example,

```
public final double X_REFERENCE = 1.234;
```

```
private final int MAX_ID = 9999;
```

```
MAX_ID = 10000;    //compilation error: cannot assign a value to final variable
MAX_ID
```

```
// You need to initialize a final member variable during declaration
```

```
private final int SIZE;    //compilation error: variable SIZE might not have
been initialized
```

Constant Naming Convention: A constant name is a noun, or a noun phrase made up of several words. All words are in uppercase separated by underscores '\_', for examples, `X_REFERENCE`, `MAX_INTEGER` and `MIN_VALUE`.

Advanced notes for `final`:

- A `final` primitive variable cannot be re-assigned a new value.
- A `final` instance cannot be re-assigned a new object.
- A `final` class cannot be sub-classed (or extended).
- A `final` method cannot be overridden.

We shall include constructors, getters, setters, `toString()`, and use the keyword `"this"`. The class diagram for the final `Circle` class is as follows:

```

1  /**
2   * The Circle class models a circle with a radius and color.
3   */
4  public class Circle {    // Save as "Circle.java"
5      // The public constants
6      public static final double DEFAULT_RADIUS = 1.0;
7      public static final String DEFAULT_COLOR = "red";
8
9      // The private instance variables
10     private double radius;
11     private String color;
12
13     // The (overloaded) constructors
14     /** Constructs a Circle with default radius and color */
15     public Circle() {          // 1st (default) Constructor
16         this.radius = DEFAULT_RADIUS;
17         this.color = DEFAULT_COLOR;
18     }
19     /** Constructs a Circle with the given radius and default color */
20     public Circle(double radius) {    // 2nd Constructor
21         this.radius = radius;
22         this.color = DEFAULT_COLOR;
23     }
24     /** Constructs a Circle with the given radius and color */
25     public Circle(double radius, String color) { // 3rd Constructor
26         this.radius = radius;
27         this.color = color;
28     }
29
30     /** Returns the radius - the public getter for private variable
31     radius. */
32     public double getRadius() {
33         return this.radius;
34     }
35     /** Sets the radius - the public setter for private variable radius
36     */
37     public void setRadius(double radius) {
38         this.radius = radius;
39     }
40     /** Returns the color - the public getter for private variable color
41     */
42     public String getColor() {
43         return this.color;
44     }
45     /** Sets the color - the public setter for private variable color */
46     public void setColor(String color) {
47         this.color = color;
48     }
49
50     /** Returns a self-descriptive string for this Circle instance */
51     public String toString() {
52         return "Circle[radius=" + radius + ", color=" + color + "];";

```

```

53     }
54
55     /** Returns the area of this Circle */
56     public double getArea() {
57         return radius * radius * Math.PI;
58     }
59
60     /** Returns the circumference of this Circle */
61     public double getCircumference() {
62         return 2.0 * radius * Math.PI;
63     }
64 }

```

### A Test Driver for the Circle Class (TestCircle.java)

```

/**
 * A Test Driver for the Circle class
 */
public class TestCircle {
    public static void main(String[] args) {
        // Test all constructors and toString()
        Circle c1 = new Circle(1.1, "blue");
        System.out.println(c1); // implicitly run toString()
        //Circle[radius=1.1, color=blue]
        Circle c2 = new Circle(2.2);
        System.out.println(c2);
        //Circle[radius=2.2, color=red]
        Circle c3 = new Circle();
        System.out.println(c3);
        //Circle[radius=1.0, color=red]

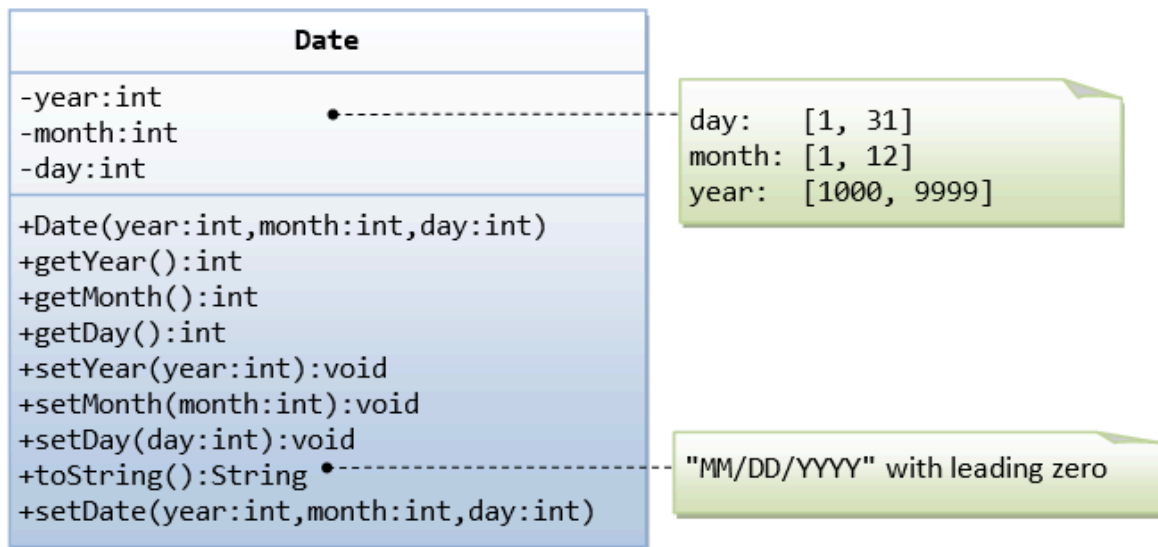
        // Test Setters and Getters
        c1.setRadius(3.3);
        c1.setColor("green");
        System.out.println(c1); // use toString() to inspect the modified
instance
        //Circle[radius=3.3, color=green]
        System.out.println("The radius is: " + c1.getRadius());
        //The radius is: 3.3
        System.out.println("The color is: " + c1.getColor());
        //The color is: green

        // Test getArea() and getCircumference()
        System.out.printf("The area is: %.2f%n", c1.getArea());
        //The area is: 34.21
        System.out.printf("The circumference is: %.2f%n",
c1.getCircumference());
        //The circumference is: 20.73
    }
}

```

}

## An exercise class Date.java



A `Date` class models a calendar date with day, month and year, is designed as shown in the class diagram. It contains the following members:

- 3 private instance variables `day`, `month`, and `year`.
- Constructors, public getters and setters for the private instance variables.
- A method `setDate()`, which sets the `day`, `month` and `year`.
- A `toString()`, which returns "`DD/MM/YYYY`", with a leading zero for `DD` and `MM` if applicable.

Write the `Date` class and a test driver to test all the public methods. No input validations are required for `day`, `month`, and `year`.

### The Date Class (Date.java)

```
/**
 * The Date class models a calendar date with day, month and year.
 * This class does not perform input validation for day, month and year.
 */
public class Date {
    // The private instance variables
    private int year, month, day;

    /** Constructs a Date instance with the given year, month and day. No
    input validation */
    public Date(int year, int month, int day) {
        this.year = year;
        this.month = month;
        this.day = day;
    }

    // The public getters/setters for the private variables
    /** Returns the year */
```



```

    public int getYear() {
        return this.year;
    }
    /** Returns the month */
    public int getMonth() {
        return this.month;
    }
    /** Returns the day */
    public int getDay() {
        return this.day;
    }
    /** Sets the year. No input validation */
    public void setYear(int year) {
        this.year = year;
    }
    /** Sets the month. No input validation */
    public void setMonth(int month) {
        this.month = month;
    }
    /** Sets the day. No input validation */
    public void setDay(int day) {
        this.day = day;
    }

    /** Returns a descriptive String in the form "MM/DD/YYYY" with leading zero
    */
    public String toString() {
        // Use built-in function String.format() to form a formatted String
        return String.format("%02d/%02d/%4d", month, day, year);
        // Specifier "0" to print leading zeros
    }

    /** Sets year, month and day. No input validation */
    public void setDate(int year, int month, int day) {
        this.year = year;
        this.month = month;
        this.day = day;
    }
}

```

=====

### A Test Driver for the Date Class (TestDate.java)

```

/**
 * A Test Driver for the Date class.
 */
public class TestDate {
    public static void main(String[] args) {
        // Test constructor and toString()
        Date d1 = new Date(2020, 2, 8);
        System.out.println(d1); // toString()
        //02/08/2020
    }
}

```

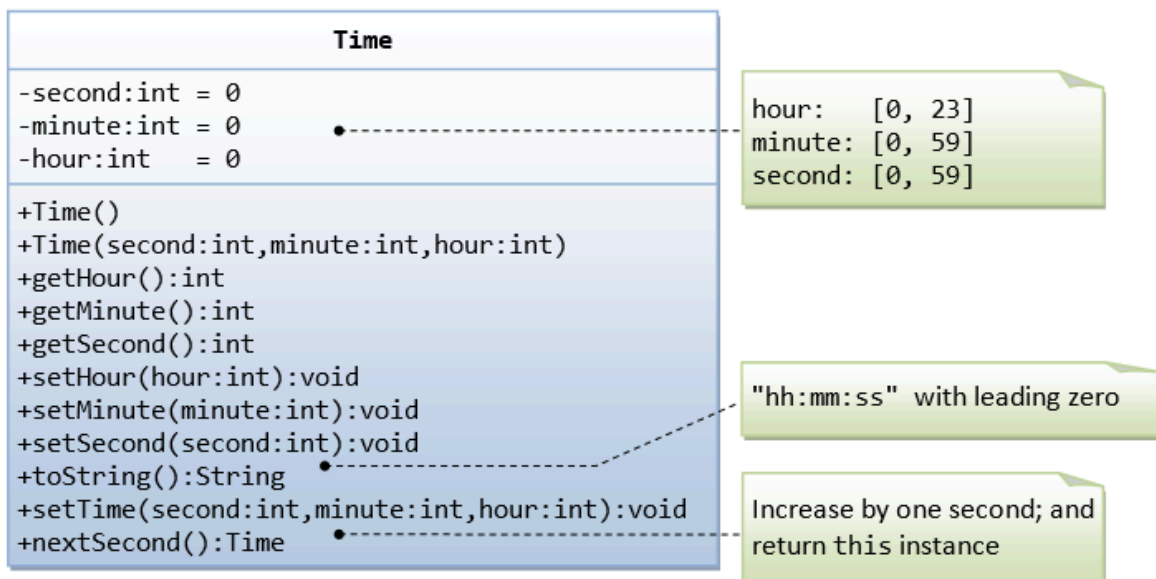
```

// Test Setters and Getters
d1.setYear(2012);
d1.setMonth(12);
d1.setDay(23);
System.out.println(d1);
//12/23/2012
System.out.println("Year is: " + d1.getYear());
//Year is: 2012
System.out.println("Month is: " + d1.getMonth());
//Month is: 12
System.out.println("Day is: " + d1.getDay());
//Day is: 23

// Test setDate()
d1.setDate(2988, 1, 2);
System.out.println(d1);
//01/02/2988
}
}

```

## Exercise (1). The Time class Time.java



A class called `Time`, which models a time instance with hour, minute and second, is designed as shown in the class diagram. It contains the following members:

- 3 private instance variables `hour`, `minute`, and `second`.
- Constructors, getters and setters.
- A method `setTime()` to set `hour`, `minute` and `second`.
- A `toString()` that returns "`hh:mm:ss`" with leading zero if applicable.
- A method `nextSecond()` that advances this instance by one second. It returns this instance to support chaining (cascading) operations, e.g.,

t1.nextSecond().nextSecond(). Take note that the nextSecond() of 23:59:59 is 00:00:00.

Write the Time class and a test driver to test all the public methods. No input validations are required.

=====

### The Time Class (Time.java)

```
/**
 * The Time class models a time instance with second, minute and hour.
 * This class does not perform input validation for second, minute and hour.
 */
public class Time {
    // The private instance variables
    private int second, minute, hour;

    // The constructors (overloaded)
    /** Constructs a Time instance with the given second, minute and hour. No
input validation */
    public Time(int second, int minute, int hour) {
        this.second = second;
        this.minute = minute;
        this.hour = hour;
    }
    /** Constructs a Time instance with the default values */
    public Time() { // the default constructor
        this.second = 0;
        this.minute = 0;
        this.hour = 0;
    }

    // The public getters/setters for the private variables.
    /** Returns the second */
    public int getSecond() {
        return this.second;
    }
    /** Returns the minute */
    public int getMinute() {
        return this.minute;
    }
    /** Returns the hour */
    public int getHour() {
        return this.hour;
    }
    /** Sets the second. No input validation */
    public void setSecond(int second) {
        this.second = second;
    }
    /** Sets the minute. No input validation */
    public void setMinute(int minute) {
        this.minute = minute;
    }
    /** Sets the hour. No input validation */
```

```

    public void setHour(int hour) {
        this.hour = hour;
    }

    /** Returns a self-descriptive string in the form of "hh:mm:ss" with
    leading zeros */
    public String toString() {
        // Use built-in function String.format() to form a formatted String
        return String.format("%02d:%02d:%02d", hour, minute, second);
        // Specifier "0" to print leading zeros, if available.
    }

    /** Sets second, minute and hour to the given values */
    public void setTime(int second, int minute, int hour) {
        // No input validation
        this.second = second;
        this.minute = minute;
        this.hour = hour;
    }

    /** Advances this Time instance by one second, and returns this instance
    to support chaining */
    public Time nextSecond() {
        ++second;
        if (second >= 60) {
            second = 0;
            ++minute;
            if (minute >= 60) {
                minute = 0;
                ++hour;
                if (hour >= 24) {
                    hour = 0;
                }
            }
        }
    }

    return this;    // Return "this" instance, to support chaining
operations
                    // e.g., t1.nextSecond().nextSecond()
    }

}

```

=====

### A Test Driver (TestTime.java)

```

/**
 * A Test Driver for the Time class
 */
public class TestTime {
    public static void main(String[] args) {
        // Test Constructors and toString()
        Time t1 = new Time(1, 2, 3);
        System.out.println(t1);    // toString()
    }
}

```

```

//03:02:01
Time t2 = new Time();    // The default constructor
System.out.println(t2);
//00:00:00

// Test Setters and Getters
t1.setHour(4);
t1.setMinute(5);
t1.setSecond(6);
    System.out.println(t1);    // run toString() to inspect the
modified instance
//04:05:06
System.out.println("Hour is: " + t1.getHour());
//Hour is: 4
System.out.println("Minute is: " + t1.getMinute());
//Minute is: 5
System.out.println("Second is: " + t1.getSecond());
//Second is: 6

// Test setTime()
t1.setTime(58, 59, 23);
System.out.println(t1);
//23:59:58

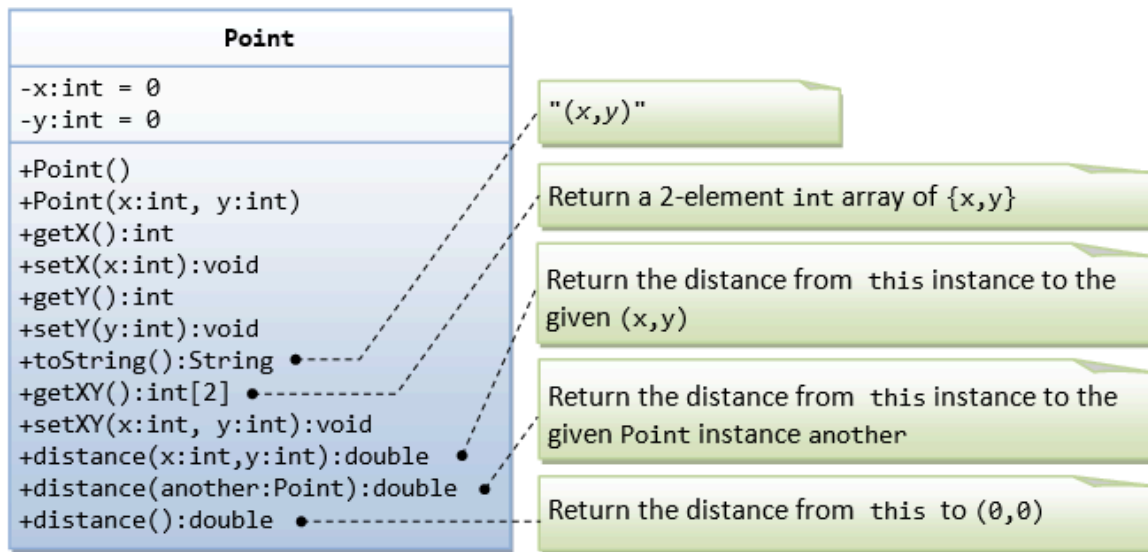
// Test nextSecond() and chaining
    System.out.println(t1.nextSecond()); // Return an instance
of Time. Invoke Time's toString()
//23:59:59

System.out.println(t1.nextSecond().nextSecond().nextSecond());    //
chaining
//00:00:02
}

}
=====

```

## Exercise (2). The Point class Point.java



A `Point` class models a 2D point at  $(x, y)$ , as shown in the class diagram. It contains the following members:

- 2 `private` instance variables `x` and `y`, which maintain the location of the point.
- Constructors, getters and setters.
- A method `setXY()`, which sets the `x` and `y` of the point; and a method `getXY()`, which returns the `x` and `y` in a 2-element `int` array.
- A `toString()`, which returns `"(x,y)"`.
- 3 versions of overloaded `distance()`:
  - `distance(int x, int y)` returns the distance from this instance to the given point at  $(x, y)$ .
  - `distance(Point another)` returns the distance from this instance to the given `Point` instance (called `another`).
  - `distance()` returns the distance from this instance to  $(0,0)$ .

=====

### The Point Class (Point.java)

```
/**
 * The Point class models a 2D point at (x, y).
 */
public class Point {
    // The private instance variables
    private int x, y;

    // The constructors (overloaded)
    /** Construct a Point instance with the default values */
    public Point() { // The default constructor
        this.x = 0;
```

```

        this.y = 0;
    }
    /** Construct a Point instance with the given x and y values */
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // The public getters and setters
    /** Returns the value of x */
    public int getX() {
        return this.x;
    }
    /** Sets the value of x */
    public void setX(int x) {
        this.x = x;
    }
    /** Returns the value of y */
    public int getY() {
        return this.y;
    }
    /** Sets the value of y */
    public void setY(int y) {
        this.y = y;
    }

    /** Returns a self-descriptive string in the form of "(x,y)" */
    public String toString() {
        return "(" + this.x + "," + this.y + ")";
    }

    /** Returns a 2-element int array containing x and y */
    public int[] getXY() {
        int[] results = new int[2];
        results[0] = this.x;
        results[1] = this.y;
        return results;
    }

    /** Sets both x and y */
    public void setXY(int x, int y) {
        this.x = x;
        this.y = y;
    }

    /** Return the distance from this instance to the given point
    at (x,y). Invoke via p1.distance(1,2) */
    public double distance(int x, int y) {

```

```

        int xDiff = this.x - x;
        int yDiff = this.y - y;
        return Math.sqrt(xDiff*xDiff + yDiff*yDiff);
    }
    /** Returns the distance from this instance to the given Point
instance. Invoke via p1.distance(p2) */
    public double distance(Point another) {
        int xDiff = this.x - another.x;
        int yDiff = this.y - another.y;
        return Math.sqrt(xDiff*xDiff + yDiff*yDiff);
    }
    /** Returns the distance from this instance to (0,0). Invoke
via p1.distance() */
    public double distance() {
        return Math.sqrt(this.x*this.x + this.y*this.y);
    }
}

```

=====

#### A Test Driver (TestPoint.java)

```

/**
 * A Test Driver for the Point class.
 */
public class TestPoint {
    public static void main(String[] args) {
        // Test constructors and toString()
        Point p1 = new Point(1, 2);
        System.out.println(p1); // toString()
        //(1,2)
        Point p2 = new Point(); // default constructor
        System.out.println(p2);
        //(0,0)

        // Test Setters and Getters
        p1.setX(3);
        p1.setY(4);
        System.out.println(p1); // run toString() to inspect the
modified instance
        //(3,4)
        System.out.println("X is: " + p1.getX());
        //X is: 3
        System.out.println("Y is: " + p1.getY());
        //Y is: 4

        // Test setXY() and getXY()
    }
}

```



```

        p1.setXY(5, 6);
        System.out.println(p1); // toString()
        //(5,6)
        System.out.println("X is: " + p1.getXY()[0]);
        //X is: 5
        System.out.println("Y is: " + p1.getXY()[1]);
        //Y is: 6

        // Test the 3 overloaded versions of distance()
        p2.setXY(10, 11);
        System.out.printf("Distance is: %.2f%n", p1.distance(10,
11));
        //Distance is: 7.07
        System.out.printf("Distance is: %.2f%n", p1.distance(p2));
        //Distance is: 7.07
        System.out.printf("Distance is: %.2f%n", p2.distance(p1));
        //Distance is: 7.07
        System.out.printf("Distance is: %.2f%n", p1.distance());
        //Distance is: 7.81
    }

}

```

In this example, we shall validate the inputs to ensure that  $0 \leq \text{hour} \leq 23$ ,  $0 \leq \text{minute} \leq 59$ , and  $0 \leq \text{second} \leq 59$ . We re-write our `Time` class as follows. Take note that all the validations are done in the setters. All other methods (such as constructors and `setTime()` invoking the setters to perform input validations - so as to avoid duplication of codes.

```

/**
 * The Time class models a time instance with second, minute and
hour.
 * This class performs input validations.
 */
public class Time {
    // The private instance variables - with input validations.
    private int second; // valid range is [0, 59]
    private int minute; // valid range is [0, 59]
    private int hour;    // valid range is [0, 23]

    // Input validations are done in the setters.
    // All the other methods (such as constructors and setTime())
invoke
    // these setters to perform input validations to avoid code
duplication.
    /** Sets the second to the given value with input validation */
    public void setSecond(int second) {
        // Input validation

```

```

        if (second < 0 || second > 59) {
            System.out.println("error: invalid second " + second);
            System.exit(1);    // non-0 for abnormal termination, 0
for normal
        }
        // Normal operation
        this.second = second;
    }
    /** Sets the minute to the given value with input validation */
    public void setMinute(int minute) {
        // Input validation
        if (minute < 0 || minute > 59) {
            System.out.println("error: invalid minute " + second);
            System.exit(1);
        }
        // Normal operation
        this.minute = minute;
    }
    /** Sets the hour to the given value with input validation */
    public void setHour(int hour) {
        // Input validation
        if (hour < 0 || hour > 59) {
            System.out.println("error: invalid hour " + second);
            System.exit(1);
        }
        // Normal operation
        this.hour = hour;
    }

    /** Sets second, minute and hour to the given values with input
validation */
    public void setTime(int second, int minute, int hour) {
        // Invoke setters to do input validation
        this.setSecond(second);
        this.setMinute(minute);
        this.setHour(hour);
    }

    /** Constructs a Time instance with the given values with input
validation */
    public Time(int second, int minute, int hour) {
        // Invoke setters to do input validation
        this.setTime(second, minute, hour);
    }
    /** Constructs a Time instance with default values */
    public Time() { // The default constructor
        this.second = 0;
        this.minute = 0;

```

```

        this.hour = 0;
    }

    // The public getters
    /** Returns the second */
    public int getSecond() {
        return this.second;
    }
    /** Returns the minute */
    public int getMinute() {
        return this.minute;
    }
    /** Returns the hour */
    public int getHour() {
        return this.hour;
    }

    /** Returns a self-descriptive string in the form of "hh:mm:ss"
    with leading zeros */
    public String toString() {
        return String.format("%02d:%02d:%02d", hour, minute,
second);
    }
    /** Advances this Time instance by one second and returns this
    instance to support chaining */
    public Time nextSecond() {
        ++second;
        if (second == 60) { // We are sure that second <= 60 here
because of the input validation
            second = 0;
            ++minute;
            if (minute == 60) {
                minute = 0;
                ++hour;
                if (hour == 24) {
                    hour = 0;
                }
            }
        }
        return this; // Return this instance, to support chaining
    }

    // Test
    public static void main(String[] args) {
        Time1 t1 = new Time1(59, 61, 25);
        // error: invalid minute 61
        // exit with exit code 1 for abnormal termination
        System.out.println(t1); // not executed
    }

```

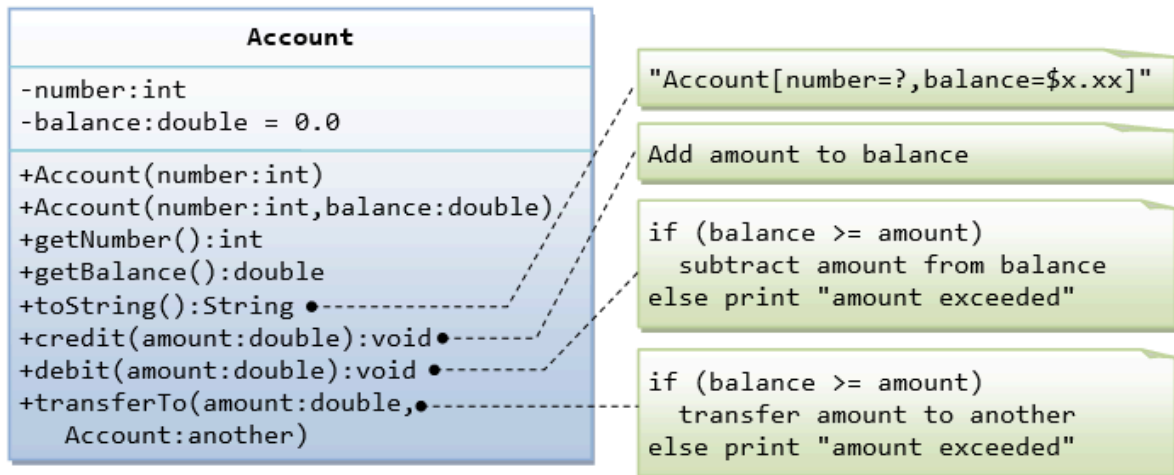
```

    }

}

```

Exercise: Write a Test Driver to test the above Time class with various valid and invalid values.



A class called `Account`, which models a bank account, is designed as shown in the class diagram. It contains the following members:

- Two private instance variables: `accountNumber` (int), and `balance` (double) which maintains the current account balance.
- Constructors (overloaded).
- Getters and Setters for the private instance variables. There is no setter for `accountNumber` as it is not designed to be changed.
- public methods `credit()` and `debit()`, which adds/subtracts the given amount to/from the balance, respectively.
- A `toString()`, which returns "A/C no:xxx, Balance=\$xxx.xx", with balance rounded to two decimal places.

Write the `Account` class and a test driver to test all the public methods.

The `Account` Class (`Account.java`)

```

1  /**
2   * The Account class models a bank account with a balance.
3   */
4  public class Account {
5      // The private instance variables
6      private int number;
7      private double balance;
8
9      // The constructors (overloaded)
10     /** Constructs an Account instance with the given number and initial
11     balance of 0 */
12     public Account(int number) {
13         this.number = number;
14         this.balance = 0.0; // "this." is optional
15     }
16     /** Constructs an Account instance with the given number and initial
17     balance */
18     public Account(int number, double balance) {
19         this.number = number;
20         this.balance = balance;
21     }
22
23     // The public getters/setters for the private instance variables.
24     // No setter for number because it is not designed to be changed.
25     // No setter for balance as it is changed via credit() and debit()
26     /** Returns the number */
27     public int getNumber() {
28         return this.number; // "this." is optional
29     }
30     /** Returns the balance */
31     public double getBalance() {
32         return this.balance; // "this." is optional
33     }
34
35     /** Returns a string description of this instance */
36     public String toString() {
37         // Use built-in function System.format() to form a formatted
38     String
39         return String.format("Account[number=%d,balance=$%.2f]", number,
40     balance);
41     }
42
43     /** Add the given amount to the balance */
44     public void credit(double amount) {
45         balance += amount;
46     }
47
48     /** Subtract the given amount from balance, if balance >= amount */
49     public void debit(double amount) {
50         if (balance >= amount) {
51             balance -= amount;
52         } else {

```

```

53         System.out.println("amount exceeded");
54     }
55 }
56
57 /** Transfer the given amount to Account another, if balance >=
58 amount */
59 public void transferTo(double amount, Account another) {
60     if (balance >= amount) {
61         this.balance -= amount;
62         another.balance += amount;
63     } else {
64         System.out.println("amount exceeded");
65     }
66 }
67 }

```

### A Test Driver for the Account Class (TestAccount.java)

```

/**
 * A Test Driver for the Account class.
 */
public class TestAccount {
    public static void main(String[] args) {
        // Test Constructors and toString()
        Account a1 = new Account(5566);
        System.out.println(a1);
        //Account[number=5566,balance=$0.00]
        Account a2 = new Account(1234, 99.9);
        System.out.println(a2);
        //Account[number=1234,balance=$99.90]

        // Test getters
        System.out.println("The account Number is: " + a2.getNumber());
        //The account Number is: 1234
        System.out.println("The balance is: " + a2.getBalance());
        //The balance is: 99.9

        // Test credit(), debit() and transferTo()
        a1.credit(11.1);
        System.out.println(a1);
        //Account[number=5566,balance=$11.10]
        a1.debit(5.5);
        System.out.println(a1);
        //Account[number=5566,balance=$5.60]
        a1.debit(500); // Test debit() error
        //amount exceeded
        System.out.println(a1);
        //Account[number=5566,balance=$5.60]
    }
}

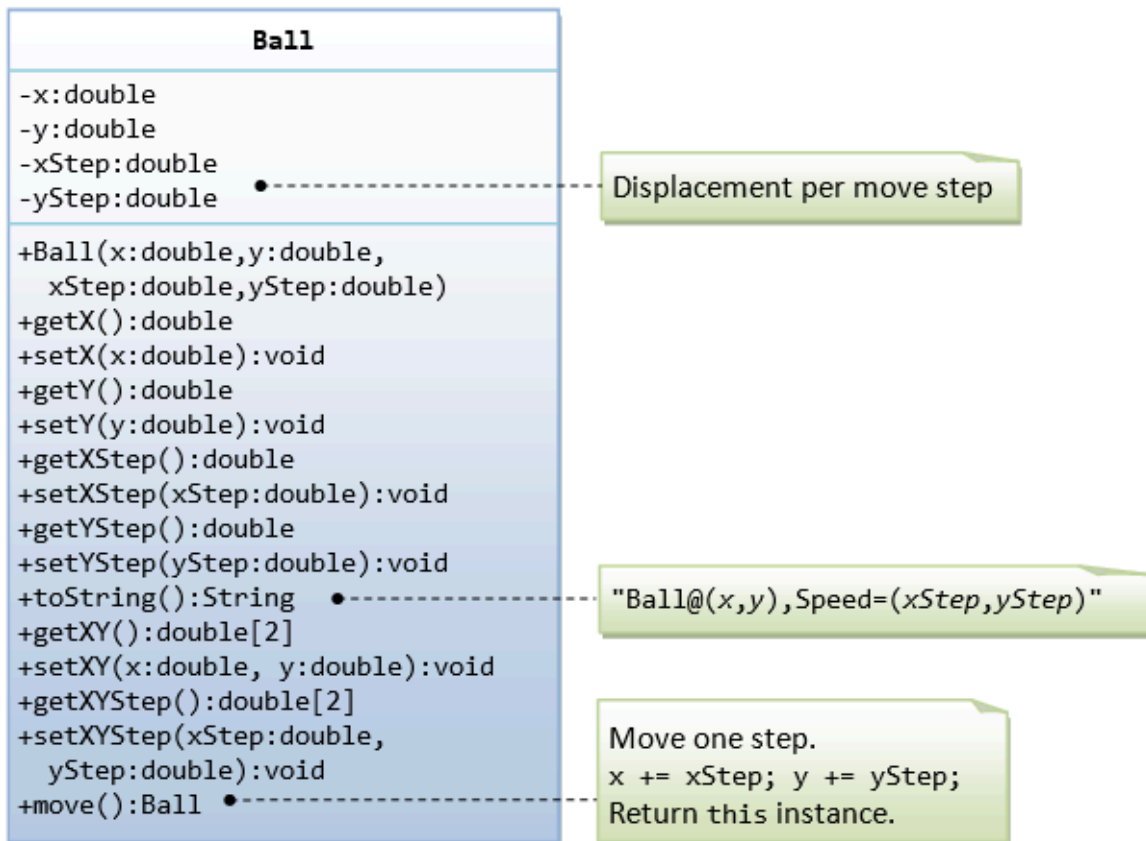
```

```

        a2.transferTo(1.0, a1);
        System.out.println(a1);
        //Account[number=5566,balance=$6.60]
        System.out.println(a2);
        //Account[number=1234,balance=$98.90]
    }
}

```

### Exercise (3). The Ball class Ball.java



A `Ball` class models a moving ball, is designed as shown in the class diagram. It contains the following members:

- 4 private variables `x`, `y`, `xStep`, `yStep`, which maintain the position of the ball and the displacement per move step.
- Constructors, getters and setters.
- Method `setXY()` and `setXYStep()`, which sets the position and step size of the ball; and `getXY()` and `getXYStep()`.
- A `toString()`, which returns `"Ball@(x,y), speed=(xStep,yStep)"`.
- A method `move()`, which increases `x` and `y` by `xStep` and `yStep` respectively; and returns `this` instance to support chaining operation.

=====

The Ball Class (Ball.java)

```

/**
 * The Ball class models a moving ball at (x, y) with displacement
 * per move-step of (xStep, yStep).
 */
public class Ball {
    // The private instance variables
    private double x, y, xStep, yStep;

    /** Constructs a Ball instance with the given input */
    public Ball(double x, double y, double xStep, double yStep) {
        this.x = x;
        this.y = y;
        this.xStep = xStep;
        this.yStep = yStep;
    }

    // The public getters and setters for the private variables
    public double getX() {
        return this.x;
    }
    public void setX(double x) {
        this.x = x;
    }
    public double getY() {
        return this.y;
    }
    public void setY(double y) {
        this.y = y;
    }
    public double getXStep() {
        return this.xStep;
    }
    public void setXStep(double xStep) {
        this.xStep = xStep;
    }
    public double getYStep() {
        return this.yStep;
    }
    public void setYStep(double yStep) {
        this.yStep = yStep;
    }

    /** Returns a self-descriptive String */
    public String toString() {
        return "Ball@(" + x + ", " + y + "), speed=(" + xStep + ", " + yStep +
")";
    }

    /** Returns the x and y position in a 2-element double array */
    public double[] getXY() {
        double[] results = new double[2];
        results[0] = this.x;
        results[1] = this.y;
    }
}

```



```

        return results;
    }
    /** Sets the x and y position */
    public void setXY(double x, double y) {
        this.x = x;
        this.y = y;
    }
    /** Returns the xStep and yStep in a 2-element double array */
    public double[] getXYStep() {
        double[] results = new double[2];
        results[0] = this.xStep;
        results[1] = this.yStep;
        return results;
    }
    /** Sets the xStep and yStep */
    public void setXYStep(double xStep, double yStep) {
        this.xStep = xStep;
        this.yStep = yStep;
    }

    /** Moves a step by increment x and y by xStep and yStep, respectively.
     * Return "this" instance to support chaining operation. */
    public Ball move() {
        x += xStep;
        y += yStep;
        return this;
    }
}

```

=====

### A Test Driver (TestBall.java)

=====

=====

```

import java.util.Arrays;
/**
 * A Test Driver for the Ball class.
 */
public class TestBall {
    public static void main(String[] args) {
        // Test constructor and toString()
        Ball b1 = new Ball(1, 2, 11, 12);
        System.out.println(b1); // toString()
        //Ball@(1.0,2.0),speed=(11.0,12.0)

        // Test Setters and Getters
        b1.setX(3);
        b1.setY(4);
        b1.setXStep(13);
        b1.setYStep(14);
        System.out.println(b1);
        //Ball@(3.0,4.0),speed=(13.0,14.0)
    }
}

```

```

        System.out.println("x is: " + b1.getX());
        //x is: 3.0
        System.out.println("y is: " + b1.getY());
        //y is: 4.0
        System.out.println("xStep is: " + b1.getXStep());
        //xStep is: 13.0
        System.out.println("yStep is: " + b1.getYStep());
        //yStep is: 14.0

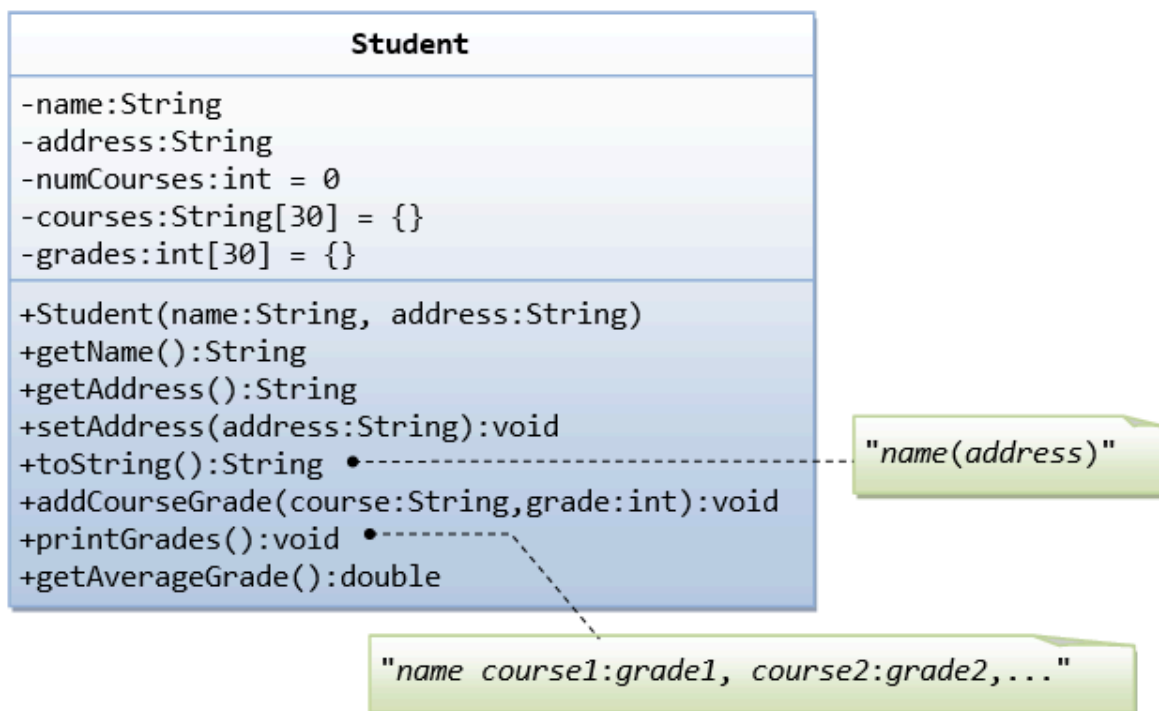
        // Test setXY(), getX(), setXYStep(), getXStep()
        b1.setXY(5, 6);
        b1.setXYStep(15, 16);
        System.out.println(b1); // toString()
        //Ball@ (5.0,6.0),speed=(15.0,16.0)
        System.out.println("x and y are: " + Arrays.toString(b1.getXY())); //
use utility to print array
        //x and y are: [5.0, 6.0]
        System.out.println("xStep    and    yStep    are:    "    +
Arrays.toString(b1.getXYStep()));
        //xStep and yStep are: [15.0, 16.0]

        // Test move() and chaining
        System.out.println(b1.move()); // toString()
        //Ball@ (20.0,22.0),speed=(15.0,16.0)
        System.out.println(b1.move().move().move());
        //Ball@ (65.0,70.0),speed=(15.0,16.0)
    }

}

```

## Exercise (4). The Student Class Student.java



Suppose that our application requires us to model students. A student has a name and an address. We are required to keep track of the courses taken by each student, together with the grades (between 0 and 100) for each of the courses. A student shall not take more than 30 courses for the entire program. We are required to print all course grades, and also the overall average grade.

We can design the `Student` class as shown in the class diagram. It contains the following members:

- private instance variables `name (String)`, `address (String)`, `numCourses (int)`, `course (String[30])` and `grades (int[30])`. The `numCourses` keeps track of the number of courses taken by this student so far. The `courses` and `grades` are two parallel arrays, storing the courses taken (e.g., `{"IM101", "IM102", "IM103"}`) and their respective grades (e.g. `{89, 56, 98}`).
- A constructor that constructs an instance with the given `name` and `Address`. It also constructs the `courses` and `grades` arrays and set the `numCourses` to 0.
- Getters for `name` and `address`; setter for `address`. No setter is defined for `name` as it is not designed to be changed.
- A `toString()`, which prints `"name (address) "`.
- A method `addCourseGrade(course, grade)`, which appends the given `course` and `grade` into the `courses` and `grades` arrays, respectively; and increments `numCourses`.
- A method `printGrades()`, which prints `"name course1:grade1, course2:grade2,..."`.
- A method `getAverageGrade()`, which returns the average grade of all the courses taken.

```
=====
=====
```

```

1  /**
2   * The student class models a student having courses and grades.
3   */
4  public class Student {
5      // The private instance variables
6      private String name;
7      private String address;
8      // The courses taken and grades for the courses are kept in 2
9  parallel arrays
10     private String[] courses;
11     private int[] grades;    // valid range is [0, 100]
12     private int numCourses;  // Number of courses taken so far
13     private static final int MAX_COURSES = 30;    // Maximum number of
14     courses taken by student
15
16     /** Constructs a Student instance with the given input */
17     public Student(String name, String address) {
18         this.name = name;
19         this.address = address;
20         courses = new String[MAX_COURSES];    // allocate arrays
21         grades = new int[MAX_COURSES];
22         numCourses = 0;                      // no courses so far
23     }
24
25     // The public getters and setters.
26     // No setter for name as it is not designed to be changed.
27     /** Returns the name */
28     public String getName() {
29         return this.name;
30     }
31     /** Returns the address */
32     public String getAddress() {
33         return this.address;
34     }
35     /** Sets the address */
36     public void setAddress(String address) {
37         this.address = address;
38     }
39
40     /** Returns a self-descriptive String */
41     public String toString() {
42         return name + "(" + address + ")";
43     }
44
45     /** Adds a course and grade */
46     public void addCourseGrade(String course, int grade) {
47         courses[numCourses] = course;
48         grades[numCourses] = grade;
49         ++numCourses;
50     }
51
52     /** Prints all courses and their grades */

```

```

53     public void printGrades() {
54         System.out.print(name);
55         for (int i = 0; i < numCourses; ++i) {
56             System.out.print(" " + courses[i] + ":" + grades[i]);
57         }
58         System.out.println();
59     }
60
61     /** Computes the average grade */
62     public double getAverageGrade() {
63         int sum = 0;
64         for (int i = 0; i < numCourses; ++i) {
65             sum += grades[i];
66         }
67         return (double)sum/numCourses;
68     }
69 }

```

### A Test Driver for the Student Class (TestStudent.java)

```

=====
/**
 * A test driver program for the Student class.
 */
public class TestStudent {
    public static void main(String[] args) {
        // Test constructor and toString()
        Student ahTeck = new Student("Tan Ah Teck", "1 Happy Ave");
        System.out.println(ahTeck); // toString()
        //Tan Ah Teck(1 Happy Ave)

        // Test Setters and Getters
        ahTeck.setAddress("8 Kg Java");
        System.out.println(ahTeck);
        //Tan Ah Teck(8 Kg Java)
        System.out.println(ahTeck.getName());
        //Tan Ah Teck
        System.out.println(ahTeck.getAddress());
        //8 Kg Java

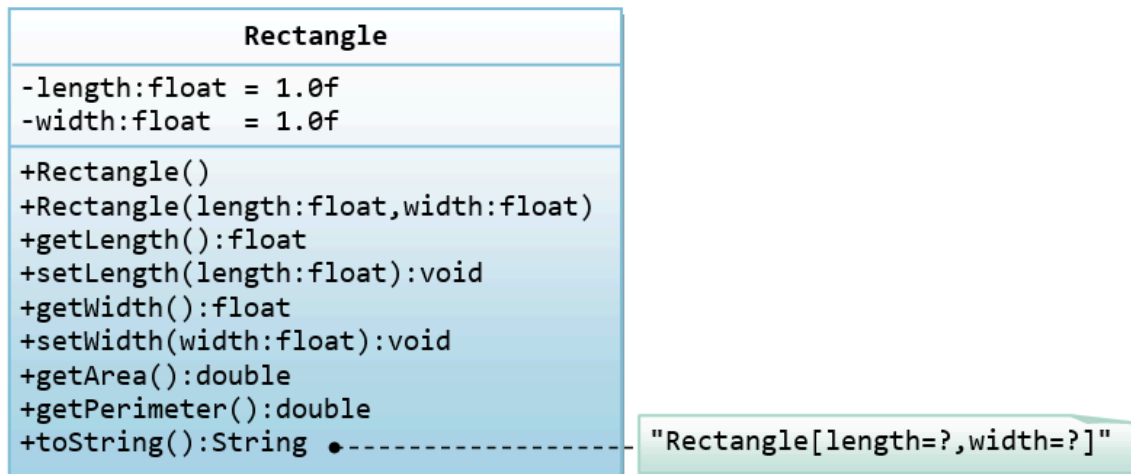
        // Test addCourseGrade(), printGrades() and getAverageGrade()
        ahTeck.addCourseGrade("IM101", 89);
        ahTeck.addCourseGrade("IM102", 57);
        ahTeck.addCourseGrade("IM103", 96);
        ahTeck.printGrades();
        //Tan Ah Teck IM101:89 IM102:57 IM103:96
        System.out.printf("The average grade is %.2f\n",
            ahTeck.getAverageGrade());
        //The average grade is 80.67
    }
}

```

}

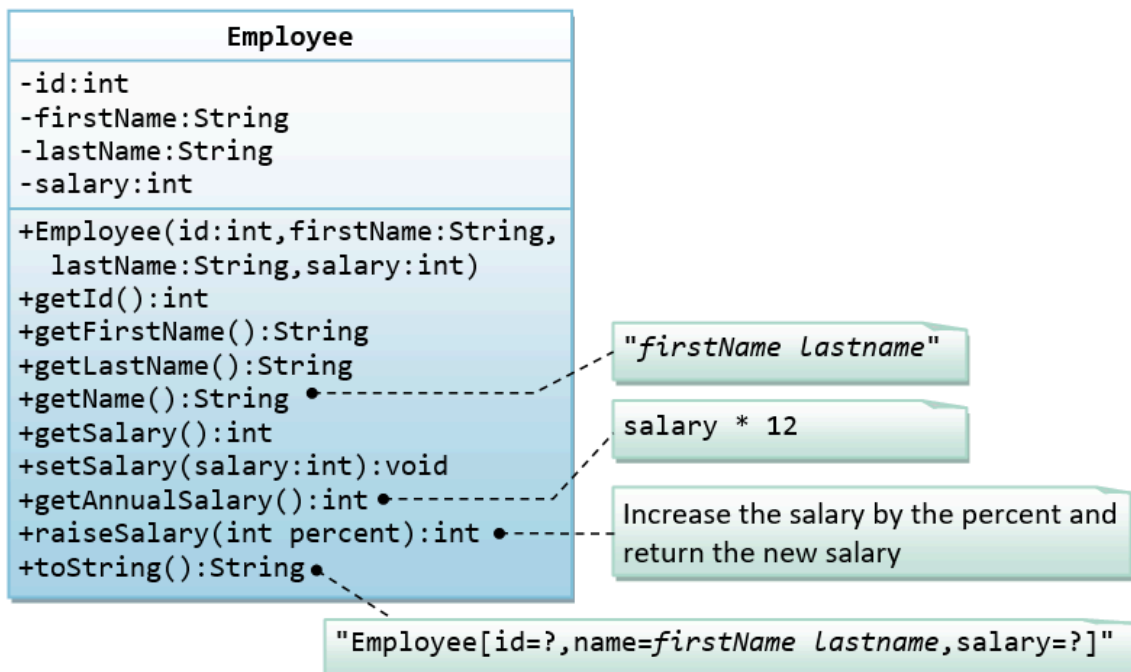
## More Exercises - Rectangle.java

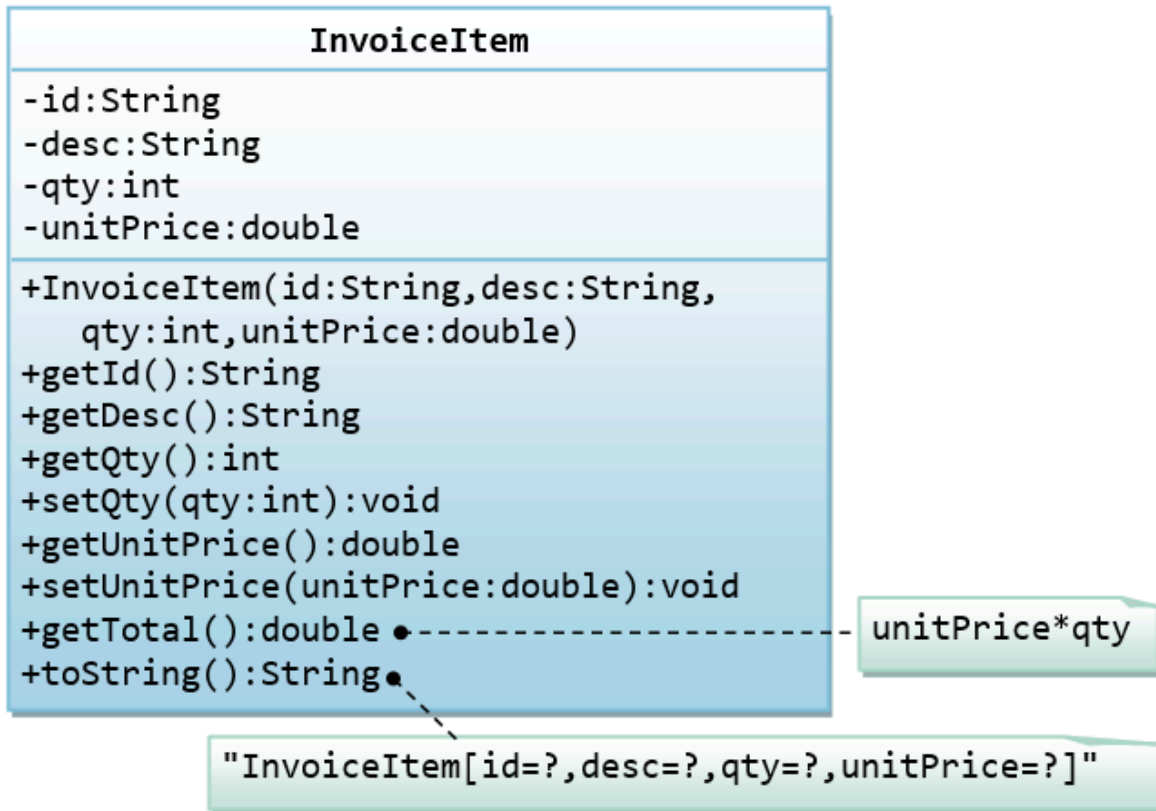
A class called `Rectangle`, which models a rectangle with a length and a width (in `float`), is designed as shown in the following class diagram. Write the `Rectangle` class.



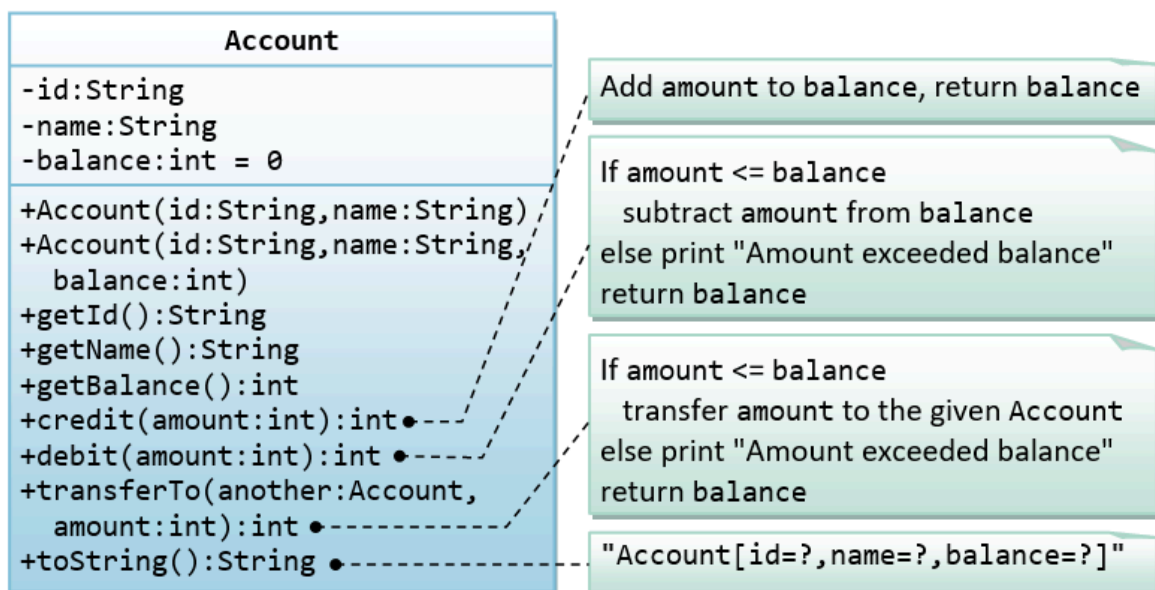
## The Employee Class

A class called `Employee`, which models an employee with an ID, name and salary, is designed as shown in the following class diagram. The method `raiseSalary(percent)` increases the salary by the given percentage. Write the `Employee` class.





A class called `Account`, which models a bank account of a customer, is designed as shown in the following class diagram. The methods `credit(amount)` and `debit(amount)` add or subtract the given amount to the balance. The method `transferTo(anotherAccount, amount)` transfers the given amount from this `Account` to the given `anotherAccount`. Write the `Account` class.



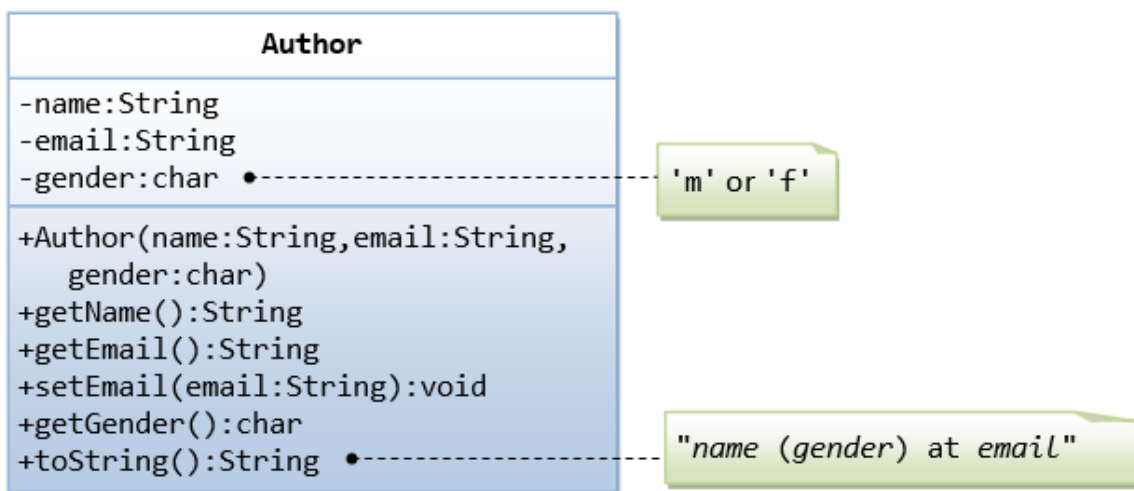
## Composition, Inheritance & Polymorphism

There are two ways to reuse existing classes, namely, composition and inheritance. With composition (aka aggregation), you define a new class, which is composed of existing classes. With inheritance, you derive a new class based on an existing class, with modifications or extensions.

### Composition

Composition EG. 1: The Author and Book Classes

Let's start with the Author class



A class called `Author` is designed as shown in the class diagram. It contains:

- Three private member variables: `name` (`String`), `email` (`String`), and `gender` (`char` of either `'m'` or `'f'` - you might also use a boolean variable called `isMale` having value of `true` or `false`).
- A constructor to initialize the `name`, `email` and `gender` with the given values.  
(There is no default constructor, as there is no default value for `name`, `email` and `gender`.)
- Public getters/setters: `getName()`, `getEmail()`, `setEmail()`, and `getGender()`.  
(There are no setters for `name` and `gender`, as these properties are not designed to be changed.)
- A `toString()` method that returns `"name (gender) at email"`, e.g., `"Tan Ah Teck (m) at ahTeck@somewhere.com"`.

**The Author Class (`Author.java`)**



```

1  /**
2   * The Author class model a book's author.
3   */
4  public class Author {
5      // The private instance variables
6      private String name;
7      private String email;
8      private char gender;    // 'm' or 'f'
9
10     /** Constructs a Author instance with the given inputs */
11     public Author(String name, String email, char gender) {
12         this.name = name;
13         this.email = email;
14         this.gender = gender;
15     }
16
17     // The public getters and setters for the private instance
18     variables.
19     // No setter for name and gender as they are not designed to be
20     changed.
21     /** Returns the name */
22     public String getName() {
23         return name;
24     }
25     /** Returns the gender */
26     public char getGender() {
27         return gender;
28     }
29     /** Returns the email */
30     public String getEmail() {
31         return email;
32     }
33     /** Sets the email */
34     public void setEmail(String email) {
35         this.email = email;
36     }
37
38     /** Returns a self-descriptive String */
39     public String toString() {
40         return name + " (" + gender + ") at " + email;
41     }
42 }

```

### A Test Driver for the Author Class (TestAuthor.java)

```

/**
 * A test driver for the Author class.
 */
public class TestAuthor {
    public static void main(String[] args) {

```

```

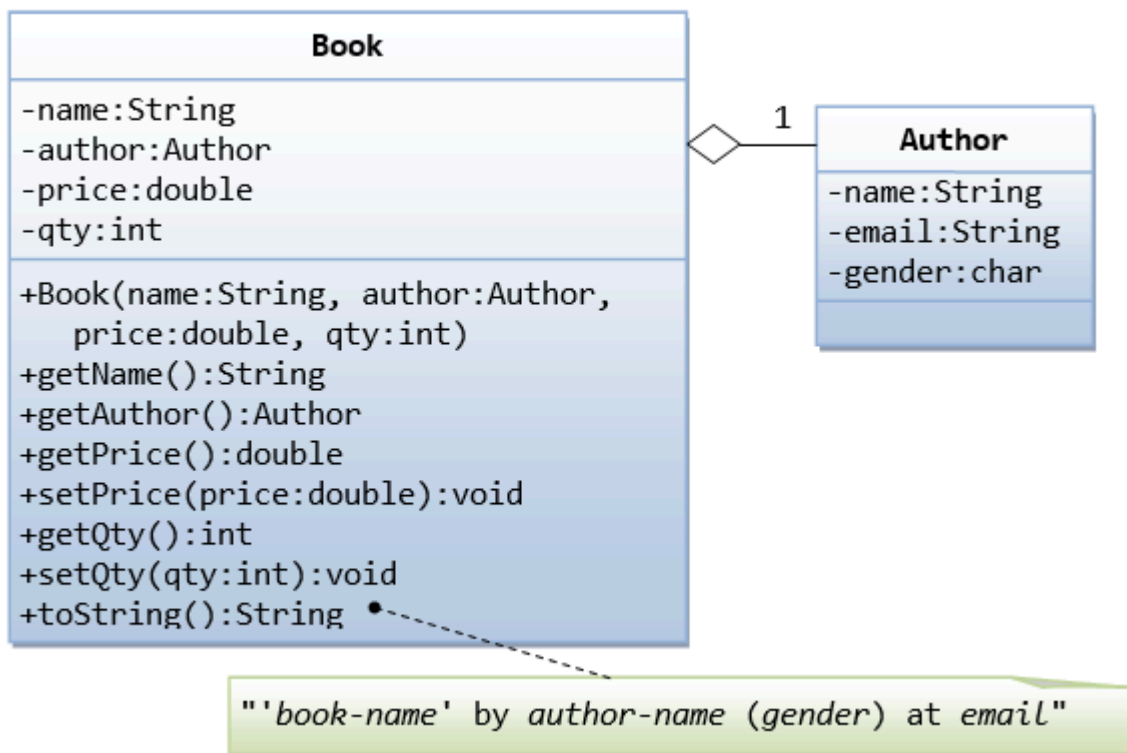
// Test constructor and toString()
Author ahTeck = new Author("Tan Ah Teck", "teck@nowhere.com", 'm');
System.out.println(ahTeck); // toString()
//Tan Ah Teck (m) at teck@nowhere.com

// Test Setters and Getters
ahTeck.setEmail("teck@somewhere.com");
System.out.println(ahTeck); // toString()
//Tan Ah Teck (m) at teck@somewhere.com
System.out.println("name is: " + ahTeck.getName());
//name is: Tan Ah Teck
System.out.println("gender is: " + ahTeck.getGender());
//gender is: m
System.out.println("email is: " + ahTeck.getEmail());
//email is: teck@somewhere.com
}

}
=====

```

A Book is written by one Author - Using an "Object" Member Variable



Let's design a `Book` class. Assume that a book is written by one (and exactly one) author. The `Book` class (as shown in the class diagram) contains the following members:

- Four private member variables: `name` (`String`), `author` (an instance of the `Author` class we have just created, assuming that each book has exactly one author), `price` (`double`), and `qty` (`int`).

- The public getters and setters: `getName()`, `getAuthor()`, `getPrice()`, `setPrice()`, `getQty()`, `setQty()`.
- A `toString()` that returns `"'book-name' by author-name (gender) at email"`. You could reuse the `Author's toString()` method, which returns `"author-name (gender) at email"`.

The Book Class (`Book.java`)

```

1  /**
2   * The Book class models a book with one (and only one) author.
3   */
4  public class Book {
5      // The private instance variables
6      private String name;
7      private Author author;
8      private double price;
9      private int qty;
10
11     /** Constructs a Book instance with the given author */
12     public Book(String name, Author author, double price, int qty) {
13         this.name = name;
14         this.author = author;
15         this.price = price;
16         this.qty = qty;
17     }
18
19     // Getters and Setters
20     /** Returns the name of this book */
21     public String getName() {
22         return name;
23     }
24     /** Return the Author instance of this book */
25     public Author getAuthor() {
26         return author; // return member author, which is an instance of
27 the class Author
28     }
29     /** Returns the price */
30     public double getPrice() {
31         return price;
32     }
33     /** Sets the price */
34     public void setPrice(double price) {
35         this.price = price;
36     }
37     /** Returns the quantity */
38     public int getQty() {
39         return qty;
40     }
41     /** Sets the quantity */
42     public void setQty(int qty) {
43         this.qty = qty;
44     }
45
46     /** Returns a self-descriptive String */
47     public String toString() {
48         return "'" + name + "' by " + author; // author.toString()
49     }
50 }

```

## A Test Driver Program for the Book Class (TestBook.java)

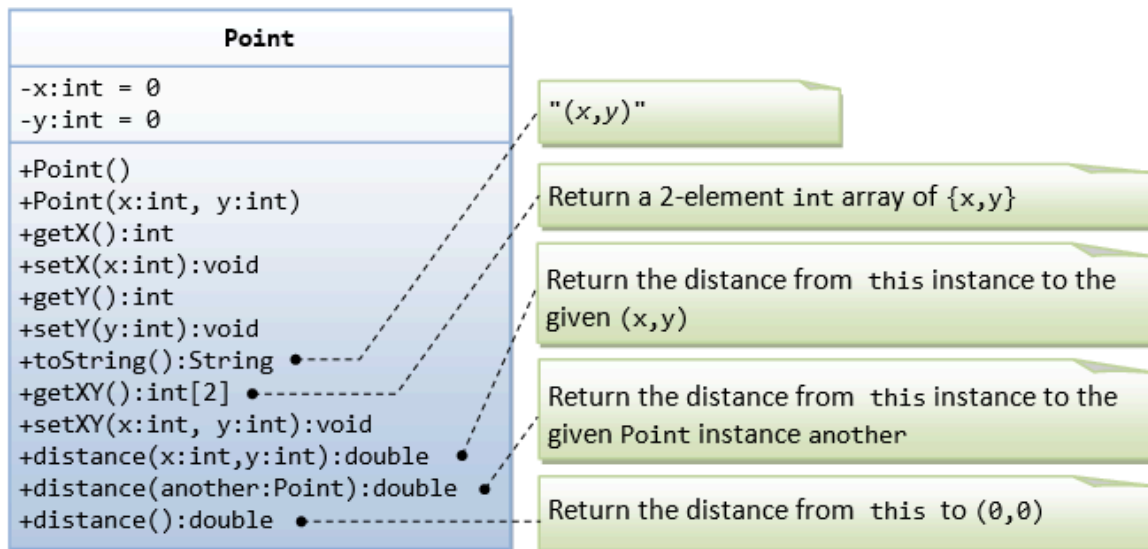
```
/**
 * A test driver program for the Book class.
 */
public class TestBook {
    public static void main(String[] args) {
        // We need an Author instance to create a Book instance
        Author ahTeck = new Author("Tan Ah Teck", "ahTeck@somewhere.com", 'm');
        System.out.println(ahTeck); // Author's toString()
        //Tan Ah Teck (m) at ahTeck@somewhere.com

        // Test Book's constructor and toString()
        Book dummyBook = new Book("Java for dummies", ahTeck, 9.99, 99);
        System.out.println(dummyBook); // Book's toString()
        //'Java for dummies' by Tan Ah Teck (m) at ahTeck@somewhere.com

        // Test Setters and Getters
        dummyBook.setPrice(8.88);
        dummyBook.setQty(88);
        System.out.println("name is: " + dummyBook.getName());
        //name is: Java for dummies
        System.out.println("price is: " + dummyBook.getPrice());
        //price is: 8.88
        System.out.println("qty is: " + dummyBook.getQty());
        //qty is: 88
        System.out.println("author is: " + dummyBook.getAuthor()); // invoke
        Author's toString()
        //author is: Tan Ah Teck (m) at ahTeck@somewhere.com
        System.out.println("author's name is: " +
dummyBook.getAuthor().getName());
        //author's name is: Tan Ah Teck
        System.out.println("author's email is: " +
dummyBook.getAuthor().getEmail());
        //author's email is: ahTeck@somewhere.com
        System.out.println("author's gender is: " +
dummyBook.getAuthor().getGender());
        //author's gender is: m

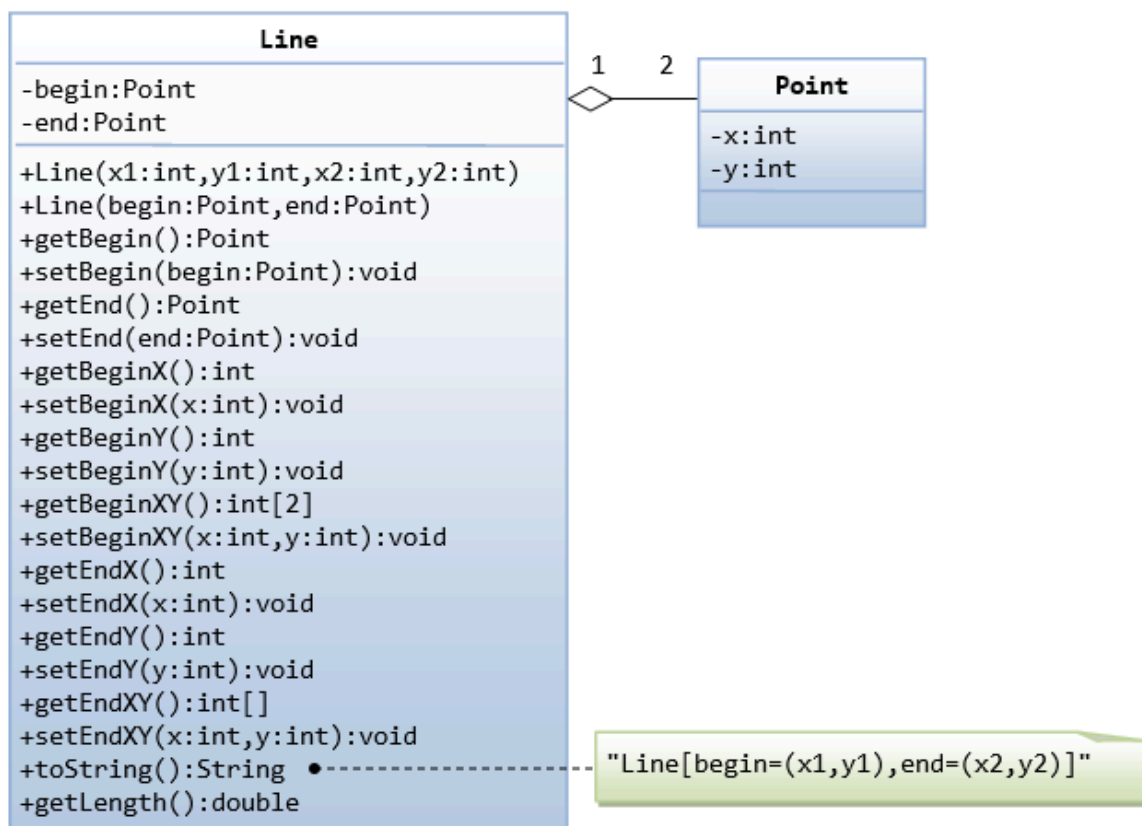
        // Using an anonymous Author instance to create a Book instance
        Book moreDummyBook = new Book("Java for more dummies",
            new Author("Peter Lee", "peter@nowhere.com", 'm'), // an
anonymous Author's instance
            19.99, 8);
        System.out.println(moreDummyBook); // Book's toString()
        //'Java for more dummies' by Peter Lee (m) at peter@nowhere.com
    }
}
```

**Notes:** In this example, I used "name" for Book class instead of "title" to illustrate that you can have a variable name in both the Author and Book classes, but they are distinct.



As an example of reusing a class via composition, suppose that we have an existing class called `Point`, defined as shown in the above class diagram. The source code is [HERE](#).

Suppose that we need a new class called `Line`, we can design the `Line` class by re-using the `Point` class via composition. We say that "A line is composed of two points", or "A line has two points". Composition exhibits a "has-a" relationship.



The Line Class via Composition (Line.java)

```

1  /**
2   * A Line composes of two Points - a begin "Point" and an end "Point".
3   * We reuse the Point class via composition.
4   */
5  public class Line {
6      // The private instance variables
7      Point begin, end;    // instances of the "Point" class
8
9      /** Constructs a Line instance given 2 points at (x1, y1) and (x2,
10     y2) */
11     public Line(int x1, int y1, int x2, int y2) {
12         begin = new Point(x1, y1); // Construct the instances declared
13         end    = new Point(x2, y2);
14     }
15     /** Construct a Line instance given 2 Point instances */
16     public Line(Point begin, Point end) {
17         this.begin = begin; // The caller had constructed the instances
18         this.end    = end;
19     }
20
21     // The public getter and setter for the private instance variables
22     public Point getBegin() {
23         return begin;
24     }
25     public Point getEnd() {
26         return end;
27     }
28     public void setBegin(Point begin) {
29         this.begin = begin;
30     }
31     public void setEnd(Point end) {
32         this.end = end;
33     }
34
35     public int getBeginX() {
36         return begin.getX(); // Point's getX()
37     }
38     public void setBeginX(int x) {
39         begin.setX(x); // Point's setX()
40     }
41     public int getBeginY() {
42         return begin.getY(); // Point's getY()
43     }
44     public void setBeginY(int y) {
45         begin.setY(y); // Point's setY()
46     }
47     public int[] getBeginXY() {
48         return begin.getXY(); // Point's getXY()
49     }
50     public void setBeginXY(int x, int y) {
51         begin.setXY(x, y); // Point's setXY()
52     }

```

```

53     public int getEndX() {
54         return end.getX(); // Point's getX()
55     }
56     public void setEndX(int x) {
57         end.setX(x); // Point's setX()
58     }
59     public int getEndY() {
60         return end.getY(); // Point's getY()
61     }
62     public void setEndY(int y) {
63         end.setY(y); // Point's setY()
64     }
65     public int[] getEndXY() {
66         return end.getXY(); // Point's getXY()
67     }
68     public void setEndXY(int x, int y) {
69         end.setXY(x, y); // Point's setXY()
70     }
71
72     /** Returns a self-descriptive String */
73     public String toString() {
74         return "Line[begin=" + begin + ",end=" + end + "];
75             // Invoke begin.toString() and end.toString()
76     }
77
78     /** Returns the length of this line */
79     public double getLength() {
80         return begin.distance(end); // use Point's distance() method
81     }
82 }

```

### A Test Driver for Line Class (TestLine.java)

```

import java.util.Arrays;
/**
 * A Test Driver for the Line class.
 */
public class TestLine {
    public static void main(String[] args) {
        // Test constructor and toString()
        Line l1 = new Line(1, 2, 3, 4);
        System.out.println(l1); // Line's toString()
        //Line[begin=(1,2),end=(3,4)]
        Line l2 = new Line(new Point(5,6), new Point(7,8)); // anonymous
        Point's instances
        System.out.println(l2); // Line's toString()
        //Line[begin=(5,6),end=(7,8)]

        // Test Setters and Getters
        l1.setBegin(new Point(11, 12));
    }
}

```



```

        l1.setEnd(new Point(13, 14));
        System.out.println(l1); // Line's toString()
        //Line[begin=(11,12),end=(13,14)]
        System.out.println("begin is: " + l1.getBegin()); // Point's
toString()
        //begin is: (11,12)
        System.out.println("end is: " + l1.getEnd()); // Point's toString()
        //end is: (13,14)

        l1.setBeginX(21);
        l1.setBeginY(22);
        l1.setEndX(23);
        l1.setEndY(24);
        System.out.println(l1); // Line's toString()
        //Line[begin=(21,22),end=(23,24)]
        System.out.println("begin's x is: " + l1.getBeginX());
        //begin's x is: 21
        System.out.println("begin's y is: " + l1.getBeginY());
        //begin's y is: 22
        System.out.println("end's x is: " + l1.getEndX());
        //end's x is: 23
        System.out.println("end's y is: " + l1.getEndY());
        //end's y is: 24

        l1.setBeginXY(31, 32);
        l1.setEndXY(33, 34);
        System.out.println(l1); // Line's toString()
        //Line[begin=(31,32),end=(33,34)]
        System.out.println("begin's x and y are: " +
Arrays.toString(l1.getBeginXY()));
        //begin's x and y are: [31, 32]
        System.out.println("end's x and y are: " +
Arrays.toString(l1.getEndXY()));
        //end's x and y are: [33, 34]

        // Test getLength()
        System.out.printf("length is: %.2f%n", l1.getLength());
        //length is: 2.83
    }

}

```

**Exercise:** Try writing these more complex methods for the `Line` class:

```

// Return the gradient of this line in radian (use Math.atan2(y, x)).
public double getGradient()

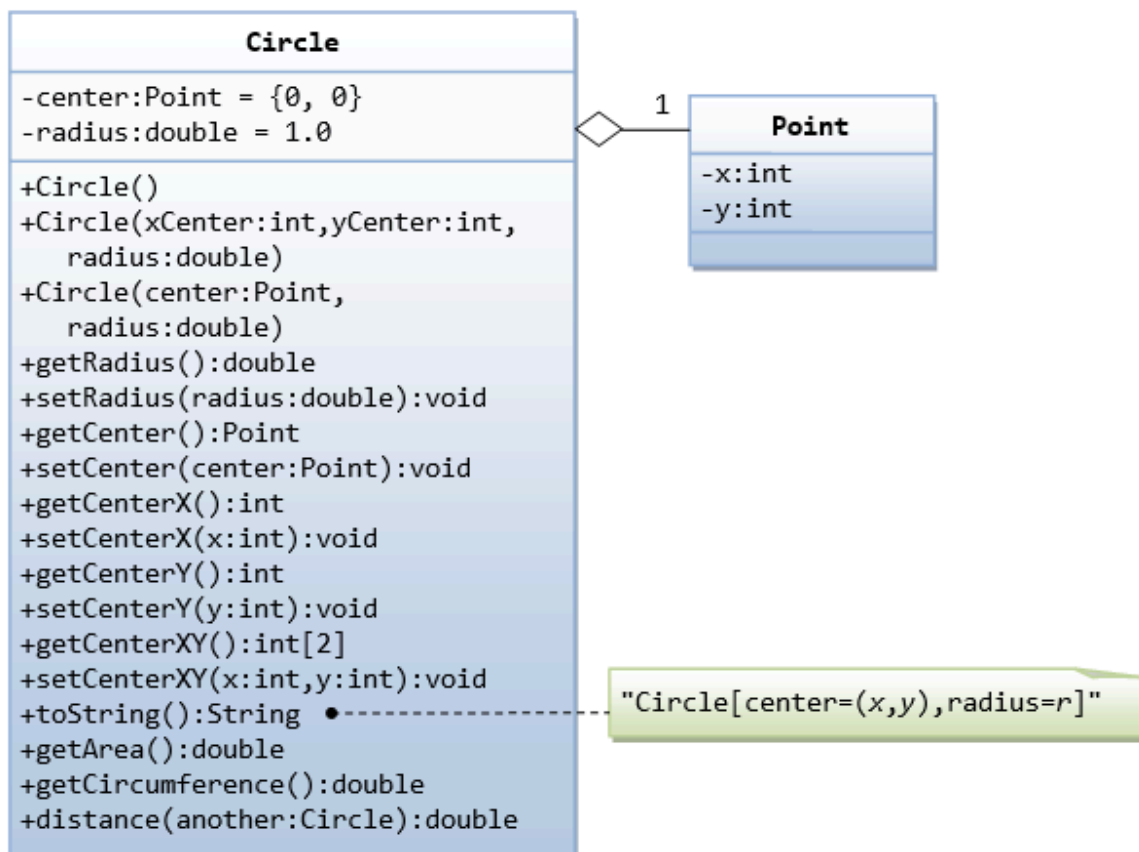
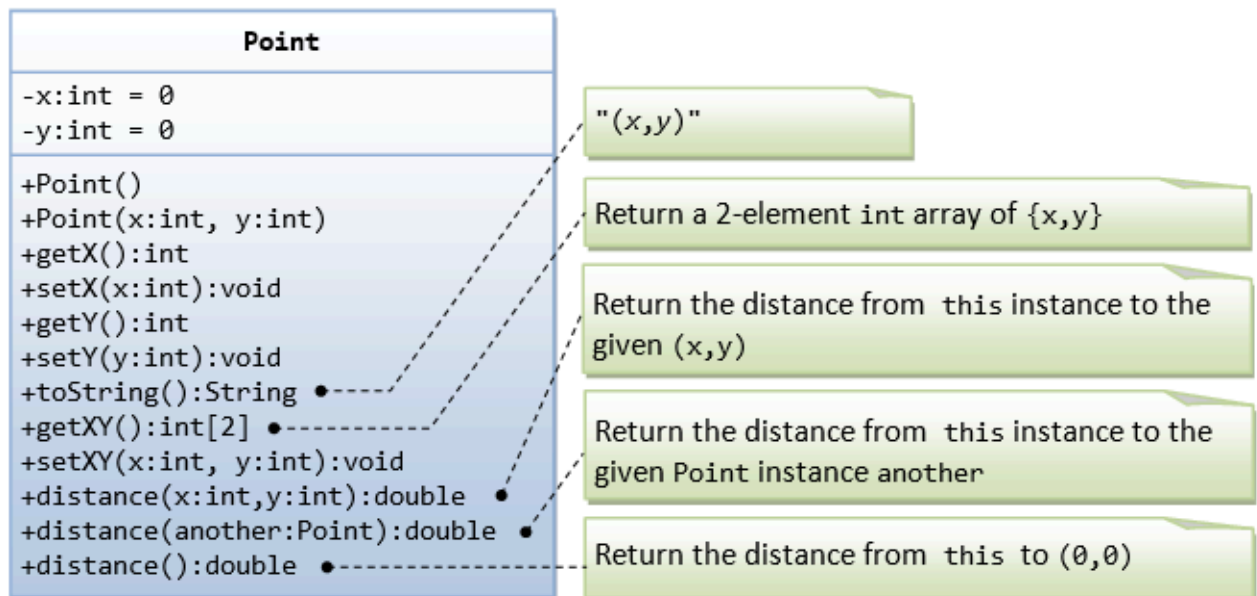
// Return the distance from this line to the given point.
public double distance(int x, int y)
public double distance(Point p)

// Return true if this line intersects the given line.

```

```
public boolean intersects(Line another)
```

=====



**It contains:**

- Two `private` member variables: a `radius` (`double`) and a `center` (an instance of `Point` class, which we created earlier).
- The constructors, `public` getters and setters.
- Methods `getCenterX()`, `setCenterX()`, `getCenterY()`, `setCenterY()`, `getCenterXY()`, `setCenterXY()`, etc.
- A `toString()` method that returns a string description of `this` instance in the format of `"Circle[center=(x,y),radius=r]"`. You should re-use the `Point`'s `toString()` to print `"(x,y)"`.
- A `distance(Circle another)` method that returns the distance from the center of `this` instance to the center of the given `Circle` instance (called `another`).

The `Circle` class (`Circle.java`)

```

1  /**
2   * The Circle class composes a "Point" instance as its center and a
3   radius.
4   * We reuse the "Point" class via composition.
5   */
6  public class Circle {
7      // The private member variables
8      private Point center; // Declare an instance of the "Point" class
9      private double radius;
10
11     // Constructors
12     /** Constructs a Circle instance with the default values */
13     public Circle() {
14         this.center = new Point(); // Construct a Point at (0,0)
15         this.radius = 1.0;
16     }
17     /** Constructs a Circle instance with the center at (xCenter,
18 yCenter) and radius */
19     public Circle(int xCenter, int yCenter, double radius) {
20         center = new Point(xCenter, yCenter); // Construct a Point at
21 (xCenter,yCenter)
22         this.radius = radius;
23     }
24     /** Constructs a Circle instance with the given Point instance as
25 center and radius */
26     public Circle(Point center, double radius) {
27         this.center = center; // The caller had constructed a Point
28 instance
29         this.radius = radius;
30     }
31
32     // Getters and Setters
33     public double getRadius() {
34         return this.radius;
35     }
36     public void setRadius(double radius) {
37         this.radius = radius;
38     }
39     public Point getCenter() {
40         return this.center; // return a Point instance
41     }
42     public void setCenter(Point center) {
43         this.center = center;
44     }
45
46     public int getCenterX() {
47         return center.getX(); // Point's getX()
48     }
49     public void setCenterX(int x) {
50         center.setX(x); // Point's setX()
51     }
52     public int getCenterY() {

```

```

53         return center.getY(); // Point's getY()
54     }
55     public void setCenterY(int y) {
56         center.setY(y); // Point's setY()
57     }
58     public int[] getCenterXY() {
59         return center.getXY(); // Point's getXY()
60     }
61     public void setCenterXY(int x, int y) {
62         center.setXY(x, y); // Point's setXY()
63     }
64
65     /** Returns a self-descriptive String */
66     public String toString() {
67         return "Circle[center=" + center + ",radius=" + radius + "]; //
68 invoke center.toString()
69     }
70
71     /** Returns the area of this circle */
72     public double getArea() {
73         return Math.PI * radius * radius;
74     }
75
76     /** Returns the circumference of this circle */
77     public double getCircumference() {
78         return 2.0 * Math.PI * radius;
79     }
80
81     /** Returns the distance from the center of this circle to the center
of
the given Circle instance called another */
    public double distance(Circle another) {
        return center.distance(another.center); // Use distance() of the
Point class
    }
}

```

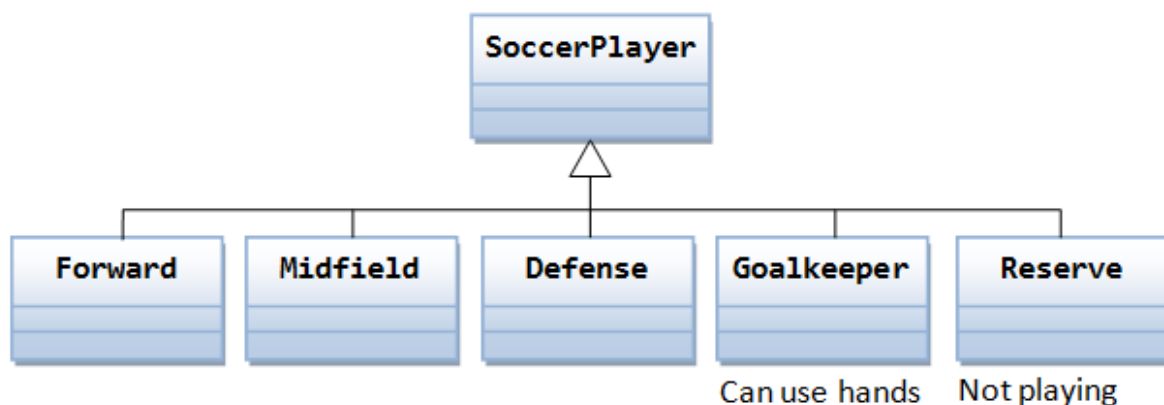
## A Test Driver for the Circle Class (TestCircle.java)

```
1  /**
2   * A test driver for the Circle class.
3   */
4  public class TestCircle {
5      public static void main(String[] args) {
6          // Test Constructors and toString()
7          Circle c1 = new Circle();
8          System.out.println(c1); // Circle's toString()
9          Circle c2 = new Circle(1, 2, 3.3);
10         System.out.println(c2); // Circle's toString()
11         Circle c3 = new Circle(new Point(4, 5), 6.6); // an anonymous
12 Point instance
13         System.out.println(c3); // Circle's toString()
14
15         // Test Setters and Getters
16         c1.setCenter(new Point(11, 12));
17         c1.setRadius(13.3);
18         System.out.println(c1); // Circle's toString()
19         System.out.println("center is: " + c1.getCenter()); // Point's
20 toString()
21         System.out.println("radius is: " + c1.getRadius());
22
23         c1.setCenterX(21);
24         c1.setCenterY(22);
25         System.out.println(c1); // Circle's toString()
26         System.out.println("center's x is: " + c1.getCenterX());
27         System.out.println("center's y is: " + c1.getCenterY());
28         c1.setCenterXY(31, 32);
29         System.out.println(c1); // Circle's toString()
30         System.out.println("center's x is: " + c1.getCenterXY()[0]);
31         System.out.println("center's y is: " + c1.getCenterXY()[1]);
32
33         // Test getArea() and getCircumference()
34         System.out.printf("area is: %.2f%n", c1.getArea());
35         System.out.printf("circumference is: %.2f%n",
36 c1.getCircumference());
37
38         // Test distance()
39         System.out.printf("distance is: %.2f%n", c1.distance(c2));
40         System.out.printf("distance is: %.2f%n", c2.distance(c1));
41     }
42 }
```

# Inheritance

---

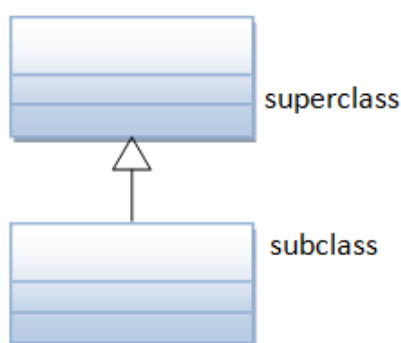
In OOP, we often organize classes in hierarchy to avoid duplication and reduce redundancy. The classes in the lower hierarchy inherit all the variables (static attributes) and methods (dynamic behaviors) from the higher hierarchies. A class in the lower hierarchy is called a subclass (or derived, child, extended class). A class in the upper hierarchy is called a superclass (or base, parent class). By pulling out all the common variables and methods into the superclasses, and leaving the specialized variables and methods in the subclasses, redundancy can be greatly reduced or eliminated as these common variables and methods do not need to be repeated in all the subclasses. For example,



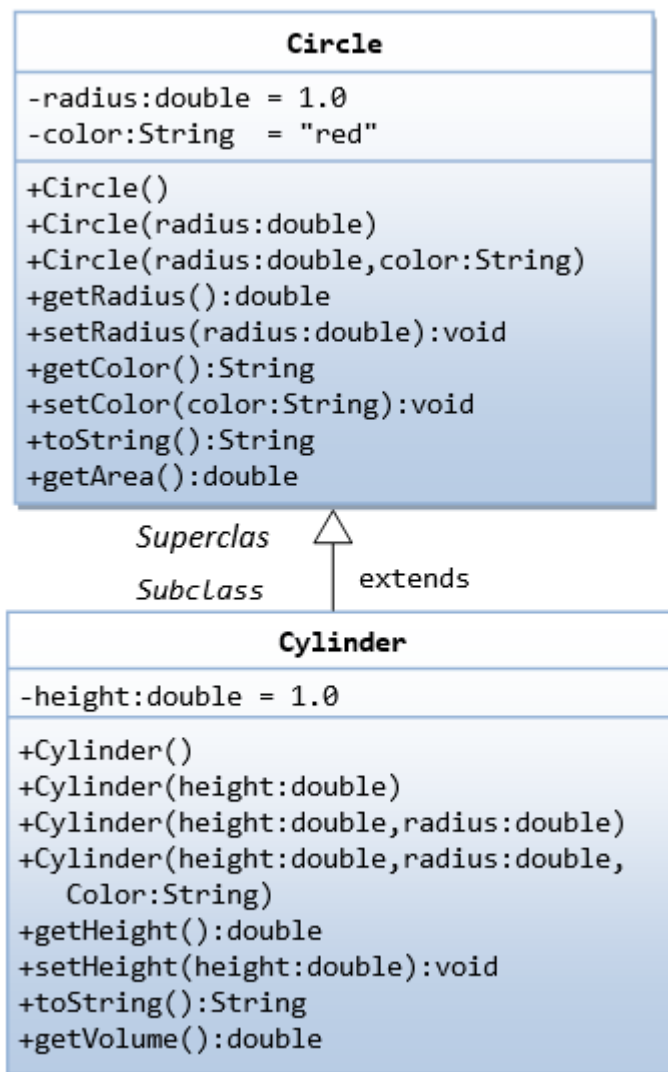
A subclass inherits all the variables and methods from its superclasses, including its immediate parent as well as all the ancestors. It is important to note that a subclass is not a "subset" of a superclass. In contrast, subclass is a "superset" of a superclass. It is because a subclass inherits all the variables and methods of the superclass; in addition, it extends the superclass by providing more variables and methods.

In Java, you define a subclass using the keyword "extends", e.g.,

```
class Goalkeeper extends SoccerPlayer {.....}
class MyApplet extends java.applet.Applet {.....}
class Cylinder extends Circle {.....}
```



## Inheritance EG. 1: The Circle and Cylinder Classes



In this example, we derive a subclass called `Cylinder` from the superclass `Circle`, which we have created in the previous chapter. It is important to note that we reuse the class `Circle`. Reusability is one of the most important properties of OOP. (Why reinvent the wheels?) The class `Cylinder` inherits all the member variables (`radius` and `color`) and methods (`getRadius()`, `getArea()`, among others) from its superclass `Circle`. It further defines a variable called `height`, two public methods - `getHeight()` and `getVolume()` and its own constructors, as shown:

Circle.java (Re-produced)

```
public class Circle {
    // private instance variables
    private double radius;
    private String color;

    // Constructors
    public Circle() {
        this.radius = 1.0;
    }
}
```



```

        this.color = "red";
        System.out.println("Constructed a Circle with Circle()"); // for
debugging
    }
    public Circle(double radius) {
        this.radius = radius;
        this.color = "red";
        System.out.println("Constructed a Circle with Circle(radius)"); // for
debugging
    }
    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
        System.out.println("Constructed a Circle with Circle(radius, color)");
// for debugging
    }

    // public getters and setters for the private variables
    public double getRadius() {
        return this.radius;
    }
    public String getColor() {
        return this.color;
    }
    public void setRadius(double radius) {
        this.radius = radius;
    }
    public void setColor(String color) {
        this.color = color;
    }

    /** Returns a self-descriptive String */
    public String toString() {
        return "Circle[radius=" + radius + ",color=" + color + "]";
    }

    /** Returns the area of this Circle */
    public double getArea() {
        return radius * radius * Math.PI;
    }
}

```

## Cylinder.java

```

/**
 * A Cylinder is a Circle plus a height.
 */
public class Cylinder extends Circle {
    // private instance variable
    private double height;

    // Constructors

```

```

    public Cylinder() {
        super(); // invoke superclass' constructor Circle()
        this.height = 1.0;
        System.out.println("Constructed a Cylinder with Cylinder()"); // for
debugging
    }
    public Cylinder(double height) {
        super(); // invoke superclass' constructor Circle()
        this.height = height;
        System.out.println("Constructed a Cylinder with Cylinder(height)"); //
for debugging
    }
    public Cylinder(double height, double radius) {
        super(radius); // invoke superclass' constructor Circle(radius)
        this.height = height;
        System.out.println("Constructed a Cylinder with Cylinder(height,
radius)"); // for debugging
    }
    public Cylinder(double height, double radius, String color) {
        super(radius, color); // invoke superclass' constructor Circle(radius,
color)
        this.height = height;
        System.out.println("Constructed a Cylinder with Cylinder(height,
radius, color)"); // for debugging
    }

    // Getter and Setter
    public double getHeight() {
        return this.height;
    }
    public void setHeight(double height) {
        this.height = height;
    }

    /** Returns the volume of this Cylinder */
    public double getVolume() {
        return getArea()*height; // Use Circle's getArea()
    }

    /** Returns a self-descriptive String */
    public String toString() {
        return "This is a Cylinder"; // to be refined later
    }
}

```

### A Test Drive for the Cylinder Class (TestCylinder.java)

```

/**
 * A test driver for the Cylinder class.
 */
public class TestCylinder {
    public static void main(String[] args) {
        Cylinder cyl = new Cylinder();
    }
}

```

```

//Constructed a Circle with Circle()
//Constructed a Cylinder with Cylinder()
System.out.println("Radius is " + cyl.getRadius()
    + ", Height is " + cyl.getHeight()
    + ", Color is " + cyl.getColor()
    + ", Base area is " + cyl.getArea()
    + ", Volume is " + cyl.getVolume());
//Radius is 1.0, Height is 1.0, Color is red,
//Base area is 3.141592653589793, Volume is 3.141592653589793

Cylinder cy2 = new Cylinder(5.0, 2.0);
//Constructed a Circle with Circle(radius)
//Constructed a Cylinder with Cylinder(height, radius)
System.out.println("Radius is " + cy2.getRadius()
    + ", Height is " + cy2.getHeight()
    + ", Color is " + cy2.getColor()
    + ", Base area is " + cy2.getArea()
    + ", Volume is " + cy2.getVolume());
//Radius is 2.0, Height is 5.0, Color is red,
//Base area is 12.566370614359172, Volume is 62.83185307179586
}

}

```

Keep the "Cylinder.java" and "TestCylinder.java" in the same directory as "Circle.class" (because we are reusing the class `Circle`). Compile and run the program.

## Method Overriding & Variable Hiding

A subclass inherits all the member variables and methods from its superclasses (the immediate parent and all its ancestors). It can use the inherited methods and variables as they are. It may also override an inherited method by providing its own version, or hide an inherited variable by defining a variable of the same name.

For example, the inherited method `getArea()` in a `Cylinder` object computes the base area of the cylinder. Suppose that we decide to override the `getArea()` to compute the surface area of the cylinder in the subclass `Cylinder`. Below are the changes:

```

1  public class Cylinder extends Circle {
2      .....
3      // Override the getArea() method inherited from superclass Circle
4      @Override
5      public double getArea() {
6          return 2*Math.PI*getRadius()*height + 2*super.getArea();
7      }
8      // Need to change the getVolume() as well
9      public double getVolume() {
10         return super.getArea()*height;    // use superclass' getArea()
11     }
12     // Override the inherited toString()
13     @Override
14     public String toString() {
15         return "Cylinder[" + super.toString() + ",height=" + height +
16     "]" + "\n";
17     }
18 }

```

If `getArea()` is called from a `Circle` object, it computes the area of the circle. If `getArea()` is called from a `Cylinder` object, it computes the surface area of the cylinder using the overridden implementation. Note that you have to use public accessor method `getRadius()` to retrieve the `radius` of the `Circle`, because `radius` is declared `private` and therefore not accessible to other classes, including the subclass `Cylinder`.

But if you override the `getArea()` in the `Cylinder`, the `getVolume()` (`=getArea()*height`) no longer works. It is because the overridden `getArea()` will be used in `Cylinder`, which does not compute the base area. You can fix this problem by using `super.getArea()` to use the superclass' version of `getArea()`. Note that `super.getArea()` can only be issued from the subclass definition, but not from an instance created, e.g. `c1.super.getArea()`, as it break the information hiding and encapsulation principle.

The "`@Override`" is known as annotation (introduced in JDK 1.5), which asks compiler to check whether there is such a method in the superclass to be overridden. This helps greatly if you misspell the name of the method to be overridden. For example, suppose that you wish to override method `toString()` in a subclass. If `@Override` is not used and `toString()` is misspelled as `TOSTRING()`, it will be treated as a new method in the subclass, instead of overriding the superclass. If `@Override` is used, the compiler will signal an error.

`@Override` annotation is optional, but certainly nice to have.

Annotations are not programming constructs. They have no effect on the program output. It is only used by the compiler, discarded after compilation, and not used by the runtime.

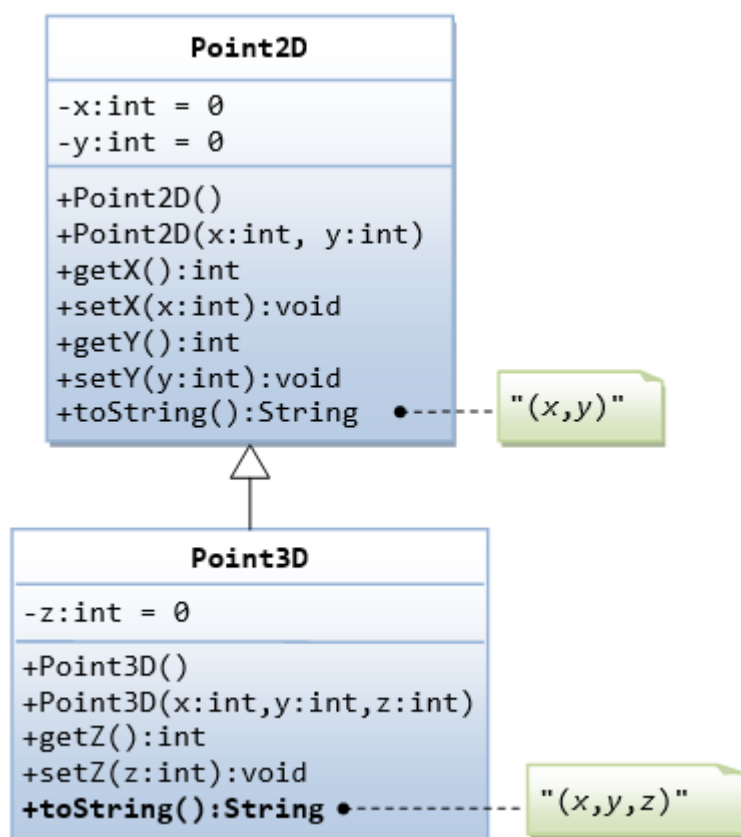
## Keyword "super"

Recall that inside a class definition, you can use the keyword `this` to refer to this instance. Similarly, the keyword `super` refers to the superclass, which could be the immediate parent or its ancestor.

The keyword `super` allows the subclass to access superclass' methods and variables within the subclass' definition. For example, `super()` and `super(argumentList)` can be used to invoke the superclass' constructor. If the subclass overrides a method inherited from its superclass, says `getArea()`, you can use `super.getArea()` to invoke the superclass' version within the subclass definition. Similarly, if your subclass hides one of the superclass' variable, you can use `super.variableName` to refer to the hidden variable within the subclass definition.

## Single Inheritance

Java does not support multiple inheritance (C++ does). Multiple inheritance permits a subclass to have more than one direct superclass. This has a serious drawback if the superclasses have conflicting implementations for the same method. In Java, each subclass can have one and only one direct superclass, i.e., single inheritance. On the other hand, a superclass can have many subclasses.



## The Superclass Point2D.java

```
/**
 * The Point2D class models a 2D point at (x, y).
 */
public class Point2D {
    // Private instance variables
    private int x, y;

    // Constructors
    /** Construct a Point2D instance at (0,0) */
    public Point2D() { // default constructor
        this.x = 0;
        this.y = 0;
    }
    /** Constructs a Point2D instance at the given (x,y) */
    public Point2D(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // Getters and Setters
    public int getX() {
        return this.x;
    }
    public void setX(int x) {
        this.x = x;
    }
    public int getY() {
        return this.y;
    }
    public void setY(int y) {
        this.y = y;
    }

    /** Returns a self-descriptive string in the form of "(x,y)" */
    @Override
    public String toString() {
        return "(" + this.x + "," + this.y + ")";
    }
}
```

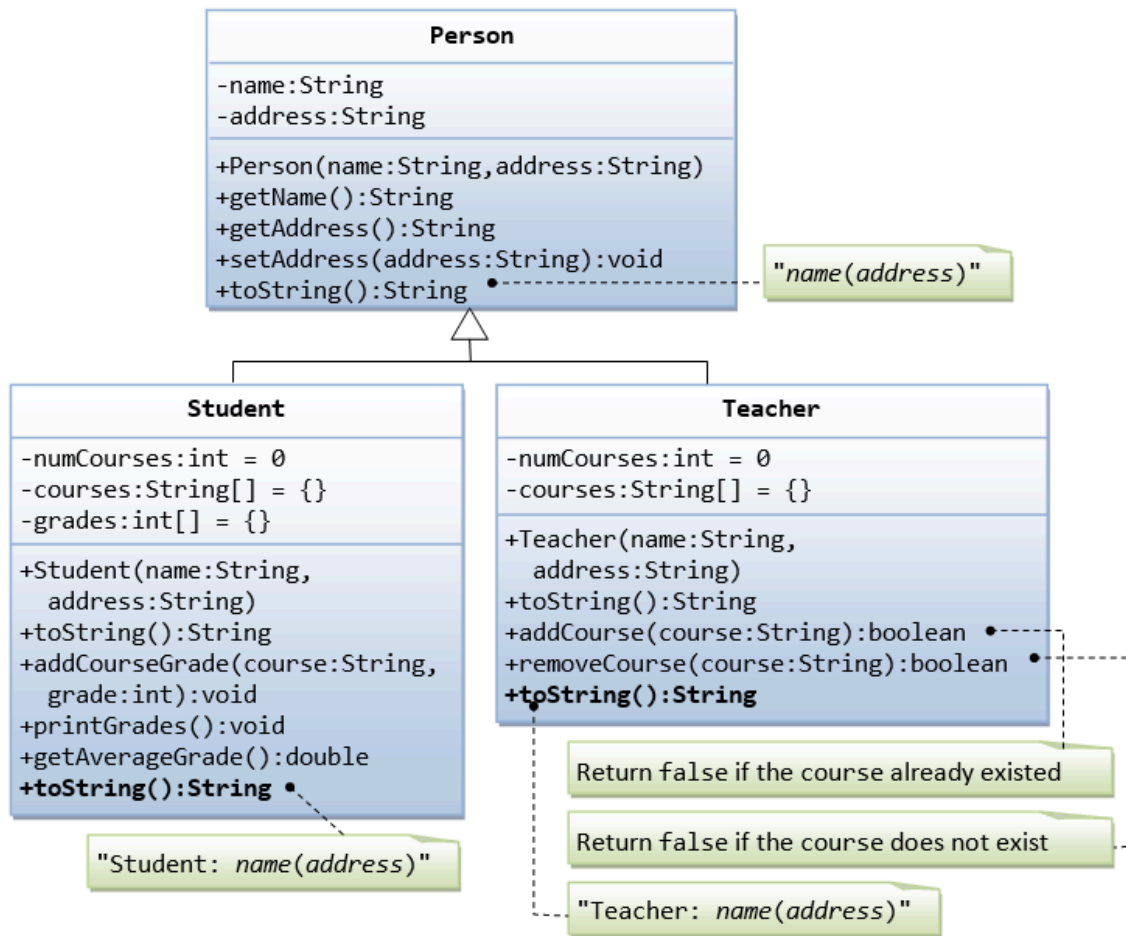
## The Subclass Point3D.java

```
1  /**
2   * The Point3D class models a 3D point at (x, y,z),
3   * which is a subclass of Point2D.
4   */
5  public class Point3D extends Point2D {
6      // Private instance variables
7      private int z;
8
9      // Constructors
10     /** Constructs a Point3D instance at (0,0,0) */
11     public Point3D() { // default constructor
12         super();      // x = y = 0
13         this.z = 0;
14     }
15     /** Constructs a Point3D instance at (x,y,z) */
16     public Point3D(int x, int y, int z) {
17         super(x, y);
18         this.z = z;
19     }
20
21     // Getters and Setters
22     public int getZ() {
23         return this.z;
24     }
25     public void setZ(int z) {
26         this.z = z;
27     }
28
29     /** Returns a self-descriptive string in the form of "(x,y,z)" */
30     @Override
31     public String toString() {
32         return "(" + super.getX() + "," + super.getY() + "," + this.z
33 + ")";
34     }
35 }
```

## A Test Driver for Point2D and Point3D Classes (TestPoint2DPoint3D.java)

```
1  /**
2   * A test driver for the Point2D and Point3D classes
3   */
4  public class TestPoint2DPoint3D {
5      public static void main(String[] args) {
6          /* Test Point2D */
7          // Test constructors and toString()
8          Point2D p2a = new Point2D(1, 2);
9          System.out.println(p2a); // toString()
10         Point2D p2b = new Point2D(); // default constructor
11         System.out.println(p2b);
12         // Test Setters and Getters
13         p2a.setX(3); // Test setters
14         p2a.setY(4);
15         System.out.println(p2a); // toString()
16         System.out.println("x is: " + p2a.getX());
17         System.out.println("x is: " + p2a.getY());
18
19         /* Test Point3D */
20         // Test constructors and toString()
21         Point3D p3a = new Point3D(11, 12, 13);
22         System.out.println(p3a); // toString()
23         Point2D p3b = new Point3D(); // default constructor
24         System.out.println(p3b);
25         // Test Setters and Getters
26         p3a.setX(21); // in superclass
27         p3a.setY(22); // in superclass
28         p3a.setZ(23); // in this class
29         System.out.println(p3a); // toString()
30         System.out.println("x is: " + p3a.getX()); // in
31         superclass
32         System.out.println("y is: " + p3a.getY()); // in
33         superclass
34         System.out.println("z is: " + p3a.getZ()); // in this
35         class
36     }
37 }
38 }
```





Suppose that we are required to model students and teachers in our application. We can define a superclass called `Person` to store common properties such as `name` and `address`, and subclasses `Student` and `Teacher` for their specific properties. For students, we need to maintain the courses taken and their respective grades; add a course with a grade, print all courses taken, and the average grade. Assume that a student takes no more than 30 courses for the entire program. For teachers, we need to maintain the courses taught currently and be able to add or remove a course taught. Assume that a teacher teaches not more than 5 courses concurrently.

We design the classes as follows:

The Superclass `Person.java`

```

/**
 * The superclass Person has name and address.
 */
public class Person {
    // private instance variables
    private String name, address;

    /** Constructs a Person instance with the given name and address */
    public Person(String name, String address) {
        this.name = name;
        this.address = address;
    }
  
```

```

    }

    // Getters and Setters
    /** Returns the name */
    public String getName() {
        return name;
    }
    /** Returns the address */
    public String getAddress() {
        return address;
    }
    /** Sets the address */
    public void setAddress(String address) {
        this.address = address;
    }

    /** Returns a self-descriptive string */
    @Override
    public String toString() {
        return name + "(" + address + ")";
    }
}

```

### The Subclass Student.java

```

/**
 * The Student class, subclass of Person.
 */
public class Student extends Person {
    // private instance variables
    private int numCourses;    // number of courses taken so far
    private String[] courses; // course codes
    private int[] grades;     // grade for the corresponding course codes
    private static final int MAX_COURSES = 30; // maximum number of courses

    /** Constructs a Student instance with the given name and address */
    public Student(String name, String address) {
        super(name, address);
        numCourses = 0;
        courses = new String[MAX_COURSES];
        grades = new int[MAX_COURSES];
    }

    /** Returns a self-descriptive string */
    @Override
    public String toString() {
        return "Student: " + super.toString();
    }

    /** Adds a course and its grade - No input validation */
    public void addCourseGrade(String course, int grade) {
        courses[numCourses] = course;
        grades[numCourses] = grade;
    }
}

```

```

        ++numCourses;
    }

    /** Prints all courses taken and their grade */
    public void printGrades() {
        System.out.print(this);
        for (int i = 0; i < numCourses; ++i) {
            System.out.print(" " + courses[i] + ":" + grades[i]);
        }
        System.out.println();
    }

    /** Computes the average grade */
    public double getAverageGrade() {
        int sum = 0;
        for (int i = 0; i < numCourses; i++ ) {
            sum += grades[i];
        }
        return (double)sum/numCourses;
    }
}

```

### The Subclass Teacher.java

```

/**
 * The Teacher class, subclass of Person.
 */
public class Teacher extends Person {
    // private instance variables
    private int numCourses;    // number of courses taught currently
    private String[] courses; // course codes
    private static final int MAX_COURSES = 5; // maximum courses

    /** Constructs a Teacher instance with the given name and address */
    public Teacher(String name, String address) {
        super(name, address);
        numCourses = 0;
        courses = new String[MAX_COURSES];
    }

    /** Returns a self-descriptive string */
    @Override
    public String toString() {
        return "Teacher: " + super.toString();
    }

    /** Adds a course. Returns false if the course has already existed */
    public boolean addCourse(String course) {
        // Check if the course already in the course list
        for (int i = 0; i < numCourses; i++) {
            if (courses[i].equals(course)) return false;
        }
        courses[numCourses] = course;
    }
}

```

```

        numCourses++;
        return true;
    }

    /** Removes a course. Returns false if the course cannot be found in the
    course list */
    public boolean removeCourse(String course) {
        boolean found = false;
        // Look for the course index
        int courseIndex = -1; // need to initialize
        for (int i = 0; i < numCourses; i++) {
            if (courses[i].equals(course)) {
                courseIndex = i;
                found = true;
                break;
            }
        }
        if (found) {
            // Remove the course and re-arrange for courses array
            for (int i = courseIndex; i < numCourses-1; i++) {
                courses[i] = courses[i+1];
            }
            numCourses--;
            return true;
        } else {
            return false;
        }
    }
}

```

### A Test Driver (TestPerson.java)

```

/**
 * A test driver for Person and its subclasses.
 */
public class TestPerson {
    public static void main(String[] args) {
        /* Test Student class */
        Student s1 = new Student("Tan Ah Teck", "1 Happy Ave");
        s1.addCourseGrade("IM101", 97);
        s1.addCourseGrade("IM102", 68);
        s1.printGrades();
        //Student: Tan Ah Teck(1 Happy Ave) IM101:97 IM102:68
        System.out.println("Average is " + s1.getAverageGrade());
        //Average is 82.5

        /* Test Teacher class */
        Teacher t1 = new Teacher("Paul Tan", "8 sunset way");
        System.out.println(t1);
        //Teacher: Paul Tan(8 sunset way)
        String[] courses = {"IM101", "IM102", "IM101"};
        for (String course: courses) {
            if (t1.addCourse(course)) {

```

```

        System.out.println(course + " added");
    } else {
        System.out.println(course + " cannot be added");
    }
}
//IM101 added
//IM102 added
//IM101 cannot be added
for (String course: courses) {
    if (tl.removeCourse(course)) {
        System.out.println(course + " removed");
    } else {
        System.out.println(course + " cannot be removed");
    }
}
//IM101 removed
//IM102 removed
//IM101 cannot be removed
}
}

```

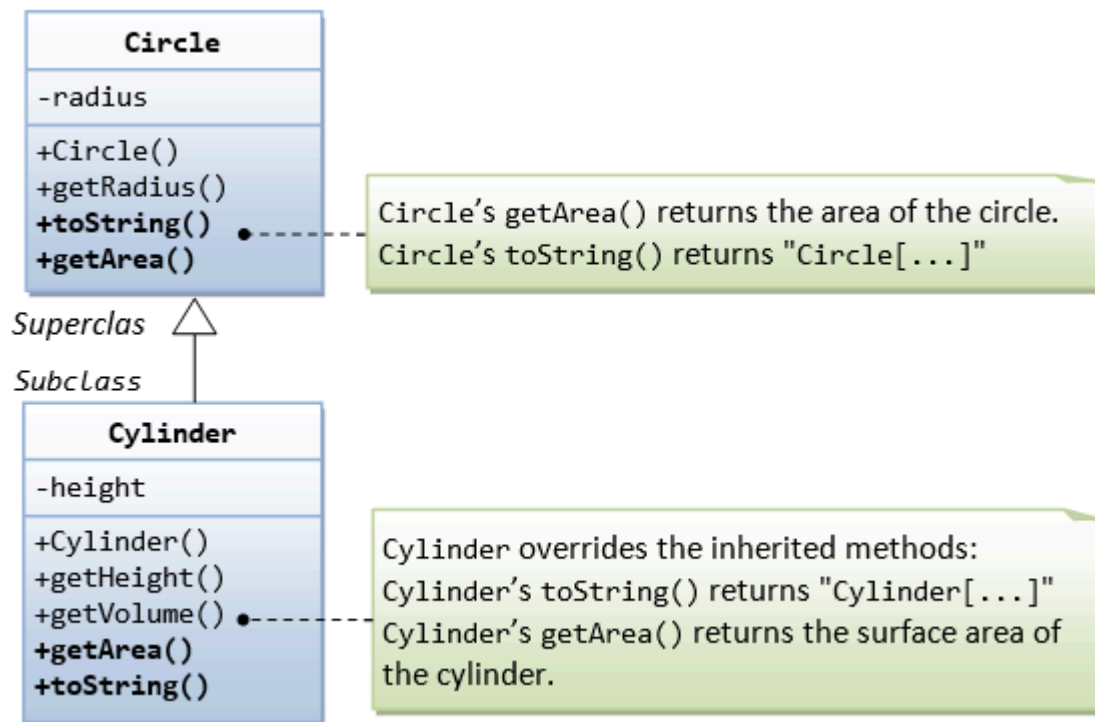
## Polymorphism

---

The word "polymorphism" means "many forms". It comes from the Greek words "poly" (means many) and "morphos" (means form). For example, in chemistry, carbon exhibits polymorphism because it can be found in more than one form: graphite and diamond. But, each of the form has it own distinct properties (and price).

## Substitutability

A subclass possesses all the attributes and operations of its superclass (because a subclass inherited all attributes and operations from its superclass). This means that a subclass object can do whatever its superclass can do. As a result, we can substitute a subclass instance when a superclass instance is expected, and everything shall work fine. This is called substitutability.



In our earlier example of `Circle` and `Cylinder`: `Cylinder` is a subclass of `Circle`. We can say that `Cylinder` "is-a" `Circle` (actually, it "is-more-than-a" `Circle`). Subclass-superclass exhibits a so called "is-a" relationship.

#### Circle.java

```

/** The superclass Circle */
public class Circle {
    // private instance variable
    private double radius;
    /** Constructs a Circle instance with the given radius */
    public Circle(double radius) {
        this.radius = radius;
    }
    /** Returns the radius */
    public double getRadius() {
        return this.radius;
    }
    /** Returns the area of this circle */
    public double getArea() {
        return radius * radius * Math.PI;
    }
    /** Returns a self-descriptive string */
    public String toString() {
        return "Circle[radius=" + radius + "]";
    }
}
  
```

#### Cylinder.java

```

/** The subclass Cylinder */
public class Cylinder extends Circle {
  
```

```

// private instance variable
private double height;
/** Constructs a Cylinder instance with the given height and radius */
public Cylinder(double height, double radius) {
    super(radius);
    this.height = height;
}
/** Returns the height */
public double getHeight() {
    return this.height;
}
/** Returns the volume of this cylinder */
public double getVolume() {
    return super.getArea() * height;
}
/** Overrides the inherited method to return the surface area */
@Override
public double getArea() {
    return 2.0 * Math.PI * getRadius() * height;
}
/** Override the inherited method to describe itself */
@Override
public String toString() {
    return "Cylinder[height=" + height + "," + super.toString() + "];"
}
}

```

Via substitutability, we can create an instance of Cylinder, and assign it to a Circle (its superclass) reference, as follows:

```

// Substitute a subclass instance to a superclass reference
Circle c1 = new Cylinder(1.1, 2.2);

```

You can invoke all the methods defined in the Circle class for the reference c1, (which is actually holding a Cylinder object), e.g.

```

// Invoke superclass Circle's methods
System.out.println(c1.getRadius());
//2.2

```

This is because a subclass instance possesses all the properties of its superclass.

However, you CANNOT invoke methods defined in the Cylinder class for the reference c1, e.g.

```

// CANNOT invoke method in Cylinder as c1 is a Circle reference
c1.getHeight();
//compilation error: cannot find symbol method getHeight()
c1.getVolume();
//compilation error: cannot find symbol method getVolume()

```

This is because c1 is a reference to the Circle class, which does not know about methods defined in the subclass Cylinder.

c1 is a reference to the Circle class, but holds an object of its subclass Cylinder. The reference c1, however, retains its internal identity. In our example, the subclass Cylinder overrides methods getArea() and toString(). c1.getArea() or c1.toString() invokes the overridden version defined in the subclass Cylinder, instead of the version defined in Circle. This is because c1 is in fact holding a Cylinder object internally.

```

System.out.println(c1.toString()); // Run the overridden version
//Cylinder[height=1.1,Circle[radius=2.2]]
System.out.println(c1.getArea()); // Run the overridden version

```

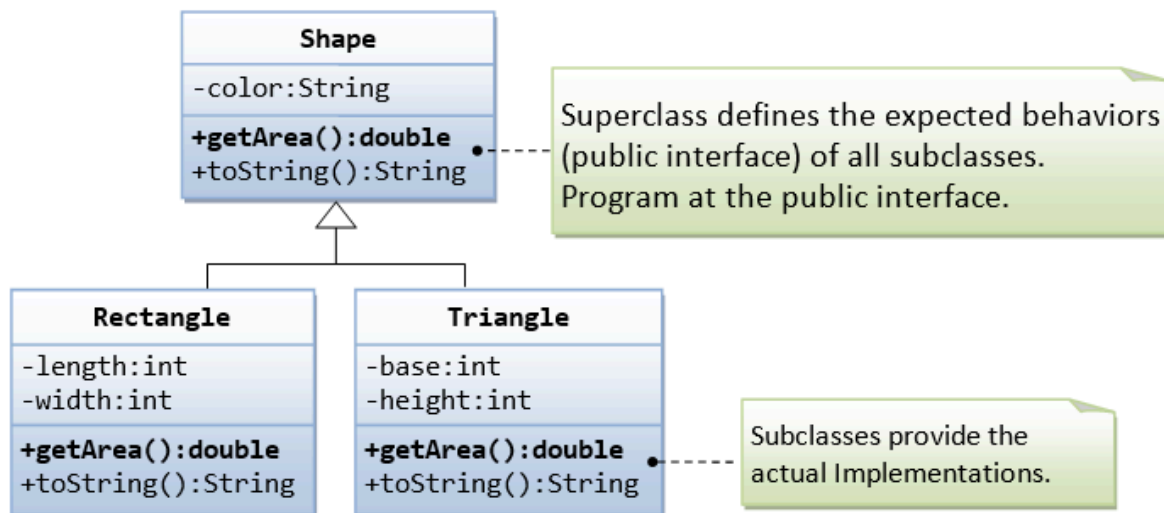
```
//15.205308443374602
```

Summary

- A subclass instance can be assigned (substituted) to a superclass' reference.
- Once substituted, we can invoke methods defined in the superclass; we cannot invoke methods defined in the subclass.
- However, if the subclass overrides inherited methods from the superclass, the subclass (overridden) versions will be invoked.

Polymorphism is very powerful in OOP to separate the interface and implementation so as to allow the programmer to program at the interface in the design of a complex system.

Consider the following example. Suppose that our program uses many kinds of shapes, such as triangle, rectangle and so on. We should design a superclass called `Shape`, which defines the public interfaces (or behaviors) of all the shapes. For example, we would like all the shapes to have a method called `getArea()`, which returns the area of that particular shape. The `Shape` class can be written as follows:.



Superclass Shape.java

```
/**
 * Superclass Shape maintain the common properties of all shapes
 */
public class Shape {
    // Private member variable
    private String color;

    /** Constructs a Shape instance with the given color */
    public Shape (String color) {
        this.color = color;
    }

    /** Returns a self-descriptive string */
    @Override
```



```

    public String toString() {
        return "Shape[color=" + color + "]\n";
    }

    /** All shapes must provide a method called getArea() */
    public double getArea() {
        // We have a problem here!
        // We need to return some value to compile the program.
        System.err.println("Shape unknown! Cannot compute area!");
        return 0;
    }
}

```

=====

Take note that we have a problem writing the `getArea()` method in the `Shape` class, because the area cannot be computed unless the actual shape is known. We shall print an error message for the time being. In the later section, I shall show you how to resolve this problem.

We can then derive subclasses, such as `Triangle` and `Rectangle`, from the superclass `Shape`.

=====

### Subclass `Rectangle.java`

```

/**
 * The Rectangle class, subclass of Shape
 */
public class Rectangle extends Shape {
    // Private member variables
    private int length, width;

    /** Constructs a Rectangle instance with the given color, length and width */
    public Rectangle(String color, int length, int width) {
        super(color);
        this.length = length;
        this.width = width;
    }

    /** Returns a self-descriptive string */
    @Override
    public String toString() {
        return "Rectangle[length=" + length + ",width=" + width + "], " +
            super.toString() + "\n";
    }

    /** Override the inherited getArea() to provide the proper implementation for rectangle */
    @Override
    public double getArea() {

```

```

        return length*width;
    }
}
=====

```

### Subclass Triangle.java

```

/**
 * The Triangle class, subclass of Shape
 */
public class Triangle extends Shape {
    // Private member variables
    private int base, height;

    /** Constructs a Triangle instance with the given color, base and height
    */
    public Triangle(String color, int base, int height) {
        super(color);
        this.base = base;
        this.height = height;
    }

    /** Returns a self-descriptive string */
    @Override
    public String toString() {
        return "Triangle[base=" + base + ",height=" + height + "," +
super.toString() + "]\n";
    }

    /** Override the inherited getArea() to provide the proper implementation
for triangle */
    @Override
    public double getArea() {
        return 0.5*base*height;
    }
}
=====

```

The subclasses override the `getArea()` method inherited from the superclass, and provide the proper implementations for `getArea()`.

### A Test Driver (TestShape.java)

In our application, we could create references of `Shape`, and assigned them instances of subclasses, as follows:

```

/**
 * A test driver for Shape and its subclasses
 */
public class TestShape {
    public static void main(String[] args) {
        Shape s1 = new Rectangle("red", 4, 5); // Upcast
        System.out.println(s1); // Run Rectangle's toString()
        //Rectangle[length=4,width=5,Shape[color=red]]
    }
}

```

```

        System.out.println("Area is " + s1.getArea()); // Run Rectangle's
getArea()
        //Area is 20.0

        Shape s2 = new Triangle("blue", 4, 5); // Upcast
        System.out.println(s2); // Run Triangle's toString()
        //Triangle[base=4,height=5,Shape[color=blue]]
        System.out.println("Area is " + s2.getArea()); // Run Triangle's
getArea()
        //Area is 10.0
    }
}
=====

```

The beauty of this code is that all the references are from the superclass (i.e., programming at the interface level). You could instantiate different subclass instance, and the code still works. You could extend your program easily by adding in more subclasses, such as `Circle`, `Square`, etc, with ease.

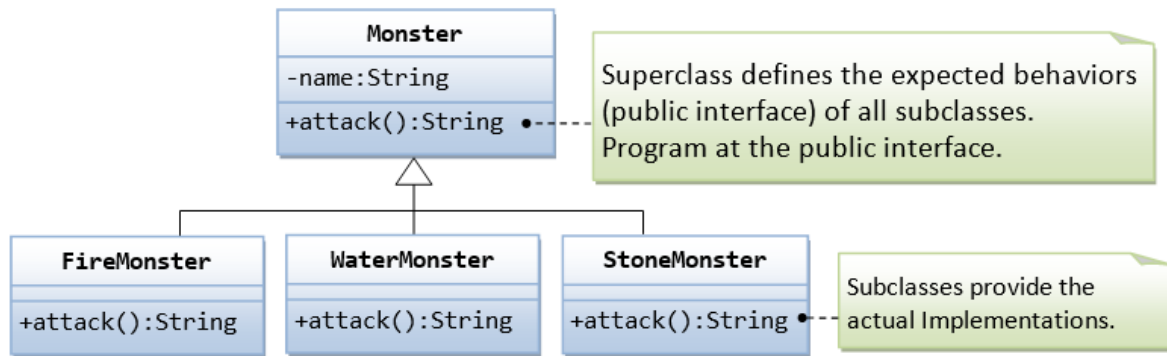
Nonetheless, the above definition of `Shape` class poses a problem, if someone instantiate a `Shape` object and invoke the `getArea()` from the `Shape` object, the program breaks.

```

=====
public class TestShape {
    public static void main(String[] args) {
        // Constructing a Shape instance poses problem!
        Shape s3 = new Shape("green");
        System.out.println(s3);
        //Shape[color=green]
        System.out.println("Area is " + s3.getArea()); // Invalid output
        //Shape unknown! Cannot compute area!
        //Area is 0.0
    }
}
=====

```

This is because the `Shape` class is meant to provide a common interface to all its subclasses, which are supposed to provide the actual implementation. We do not want anyone to instantiate a `Shape` instance. This problem can be resolved by using the so-called `abstract class`.



Polymorphism is a powerful mechanism in OOP to separate the interface and implementation so as to allow the programmer to program at the interface in the design of a complex system. For example, in our game app, we have many types of monsters that can attack. We shall design a superclass called `Monster` and define the method `attack()` in the superclass. The subclasses shall then provide their actual implementation. In the main program, we declare instances of superclass, substituted with actual subclass; and invoke method defined in the superclass.

Superclass `Monster.java`

```

/**
 * The superclass Monster defines the expected common behaviors for its
 * subclasses.
 */
public class Monster {
    // private instance variable
    private String name;

    /** Constructs a Monster instance with the given name */
    public Monster(String name) {
        this.name = name;
    }

    /** Defines a common behavior called attack() for all its subclasses */
    public String attack() {
        return "!^_&^$@+*$ I don't know how to attack!";
        // We have a problem here!
        // We need to return a String; else, compilation error!
    }
}
  
```

Subclass `FireMonster.java`

```

public class FireMonster extends Monster {
    /** Constructs a FireMonster with the given name */
    public FireMonster(String name) {
        super(name);
    }
    /** Subclass provides actual implementation for attack() */
    @Override
    public String attack() {
  
```

```

        return "Attack with fire!";
    }

}

```

### Subclass WaterMonster.java

```

public class WaterMonster extends Monster {
    /** Constructs a WaterMonster instance with the given name */
    public WaterMonster(String name) {
        super(name);
    }
    /** Subclass provides actual implementation for attack() */
    @Override
    public String attack() {
        return "Attack with water!";
    }
}

```

### Subclass StoneMonster.java

```

public class StoneMonster extends Monster {
    /** Constructs a StoneMonster instance with the given name */
    public StoneMonster(String name) {
        super(name);
    }
    /** Subclass provides actual implementation for attack() */
    @Override
    public String attack() {
        return "Attack with stones!";
    }
}

```

### A Test Driver TestMonster.java

```

public class TestMonster {
    public static void main(String[] args) {
        // Program at the specification defined in the superclass/interface.
        // Declare instances of the superclass, substituted by subclasses.
        Monster m1 = new FireMonster("r2u2");    // upcast
        Monster m2 = new WaterMonster("u2r2");   // upcast
        Monster m3 = new StoneMonster("r2r2");   // upcast

        // Invoke the actual implementation
        System.out.println(m1.attack());    // Run FireMonster's attack()
        //Attack with fire!
        System.out.println(m2.attack());    // Run WaterMonster's attack()
        //Attack with water!
        System.out.println(m3.attack());    // Run StoneMonster's attack()
        //Attack with stones!

        // m1 dies, generate a new instance and re-assign to m1.
        m1 = new StoneMonster("a2b2");    // upcast
    }
}

```

```

        System.out.println(m1.attack()); // Run StoneMonster's attack()
        //Attack with stones!

        // We have a problem here!!!
        Monster m4 = new Monster("u2u2");
        System.out.println(m4.attack()); // garbage!!!
        //!^_&^$@+%$* I don't know how to attack!
    }

}

```

## Abstract Classes

In the above examples of `Shape` and `Monster`, we encountered a problem when we created instances of `Shape` and `Monster` and run the `getArea()` or `attack()`. This can be resolved via `abstract method` and `abstract class`.

An `abstract method` is a method with only a signature (i.e., the method name, the list of arguments, and the return type) without implementation (i.e., the method's body). You use the keyword `abstract` to declare an `abstract method`.

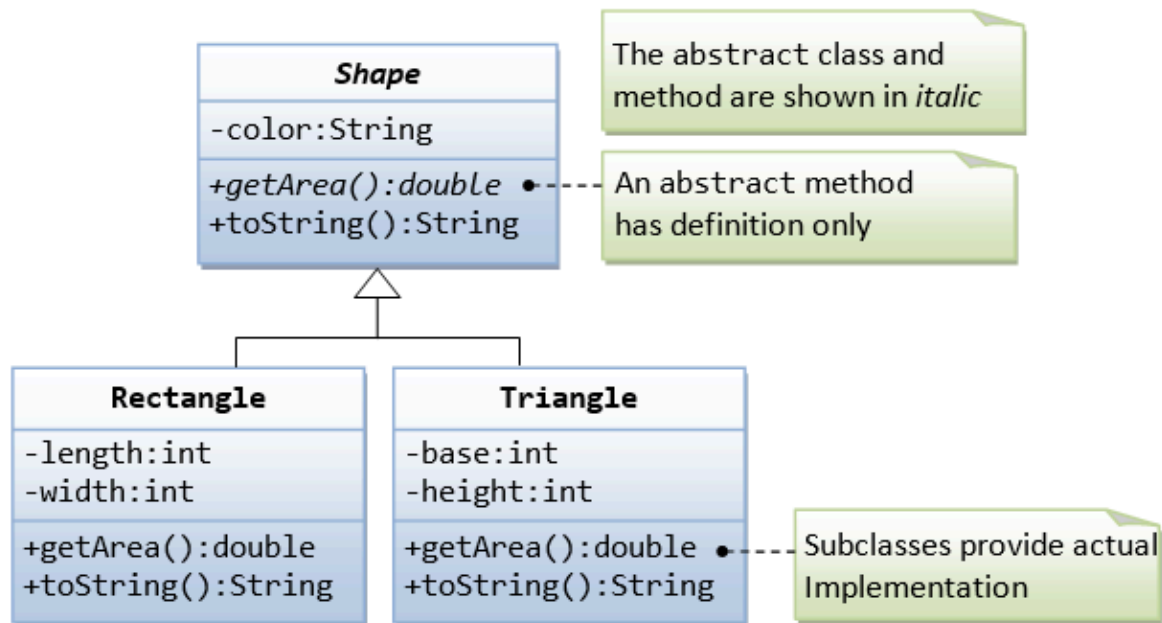
For example, in the `Shape` class, we can declare `abstract methods` `getArea()`, `draw()`, etc, as follows:

```

abstract public class Shape {
    .....
    .....
    abstract public double getArea();
    abstract public double getPerimeter();
    abstract public void draw();
}

```

Implementation of these methods is NOT possible in the `Shape` class, as the actual shape is not yet known. (How to compute the area if the shape is not known?) Implementation of these `abstract methods` will be provided later, once the actual shape is known. These `abstract methods` cannot be invoked because they have no implementation.



A class containing one or more abstract methods is called an abstract class. An abstract class must be declared with a class-modifier `abstract`. An abstract class CANNOT be instantiated, as its definition is not complete.

Let us rewrite our `Shape` class as an abstract class, containing an abstract method `getArea()` as follows:

The abstract Superclass `Shape.java`

```

/**
 * This abstract superclass Shape contains an abstract method
 * getArea(), to be implemented by its subclasses.
 */
abstract public class Shape {
    // Private member variable
    private String color;

    /** Constructs a Shape instance with the given color */
    public Shape (String color) {
        this.color = color;
    }

    /** Returns a self-descriptive string */
    @Override
    public String toString() {
        return "Shape[color=" + color + "]";
    }

    /** All Shape's concrete subclasses must implement a method called
    getArea() */
    abstract public double getArea();
}
  
```

An abstract class is incomplete in its definition since the implementation of its abstract methods is missing. Therefore, an abstract class cannot be instantiated. In other words, you cannot create instances from an abstract class (otherwise, you will have an incomplete instance with a missing method's body).

To use an abstract class, you have to derive a subclass from the abstract class. In the derived subclass, you have to override the abstract methods and provide implementation to all the abstract methods. The subclass derived is now complete, and can be instantiated. (If a subclass does not provide implementation to all the abstract methods of the superclass, the subclass remains abstract.)

This property of the abstract class solves our earlier problem. In other words, you can create instances of the subclasses such as Triangle and Rectangle, and upcast them to Shape (so as to program and operate at the interface level), but you cannot create instances of Shape, which avoid the pitfall that we have faced. For example,

```
public class TestShape {
    public static void main(String[] args) {
        Shape s1 = new Rectangle("red", 4, 5);
        System.out.println(s1);
        System.out.println("Area is " + s1.getArea());

        Shape s2 = new Triangle("blue", 4, 5);
        System.out.println(s2);
        System.out.println("Area is " + s2.getArea());

        // Cannot create instance of an abstract class
        Shape s3 = new Shape("green");
        // compilation error: Shape is abstract; cannot be instantiated
    }
}
```

=====

In summary, an abstract class provides a template for further development. The purpose of an abstract class is to provide a common interface (or protocol, or contract, or understanding, or naming convention) to all its subclasses.

For example, in the abstract class Shape, you can define abstract methods such as getArea() and draw().

No implementation is possible because the actual shape is not known. However, by specifying the signature of the abstract methods, all the subclasses are forced to use these methods' signature.

The subclasses could provide the proper implementations. Coupled with polymorphism, you can upcast subclass instances to Shape, and program at the Shape level, i.e., program at the interface.

The separation of interface and implementation enables better software design, and ease in expansion. For example, Shape defines a method called getArea(), which all the subclasses must provide



the correct implementation. You can ask for a `getArea()` from any subclasses of `Shape`, the correct area will be computed.

Furthermore, your application can be extended easily to accommodate new shapes (such as `Circle` or `Square`) by deriving more subclasses.

=====

Notes:

- An abstract method cannot be declared `final`, as `final` method cannot be overridden. An abstract method, on the other hand, must be overridden in a descendant before it can be used.
- An abstract method cannot be `private` (which generates a compilation error). This is because `private` methods are not visible to the subclass and thus cannot be overridden.

=====

We shall define the superclass `Monster` as an abstract class, containing an abstract method `attack()`. The abstract class cannot be instantiated (i.e., creating instances).

```
/**
 * The abstract superclass Monster defines the expected common behaviors,
 * via abstract methods.
 */

abstract public class Monster {
    private String name; // private instance variable
    /** Constructs a Monster instance of the given name */
    public Monster(String name) {
        this.name = name;
    }

    /** Define common behavior for all its subclasses */
    abstract public String attack();
}
```

## Java's Interface

---

A Java `interface` is a 100% abstract superclass that defines a set of methods its subclasses must support.

An `interface` contains only `public` abstract methods (methods with signatures and no implementation) and possibly constants (`public static final` variables).

You have to use the keyword `"interface"` to define an `interface` (instead of the keyword `"class"` for normal classes).

The keyword `public` and `abstract` are not needed for its abstract methods as they are mandatory.

Similar to an `abstract` superclass, an `interface` cannot be instantiated. You have to create a "subclass" that implements an interface and provides the actual implementation of all the `abstract` methods.

Unlike a normal class, where you use the keyword `"extends"` to derive a subclass. For the interface, we use the keyword `"implements"` to derive a subclass.

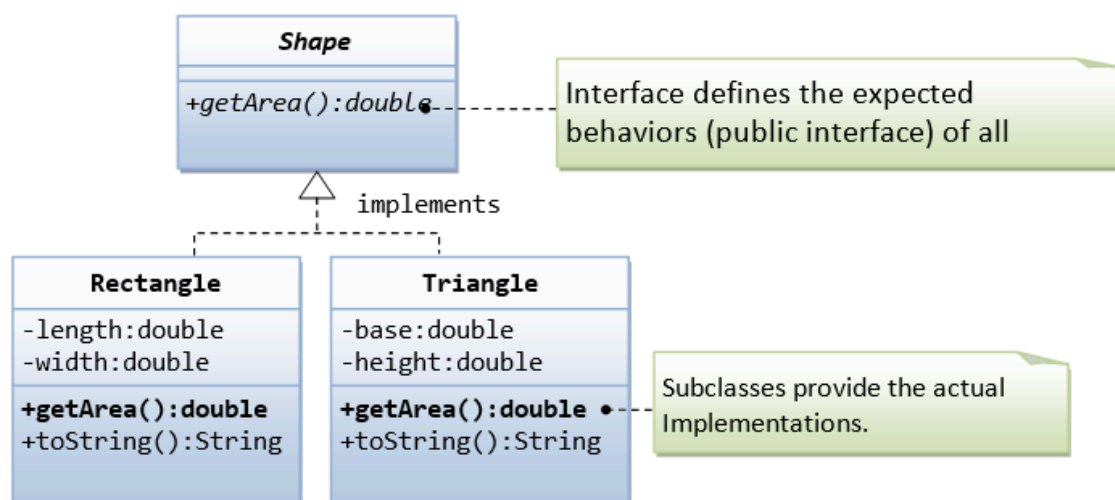
An interface is a contract for what the classes can do. It, however, does not specify how the classes should do it.

An interface provides a form, a protocol, a standard, a contract, a specification, a set of rules, an interface for all objects that implement it. It is a specification and rules that any object implementing it agrees to follow.

In Java, `abstract` class and `interface` are used to separate the public interface of a class from its implementation to allow the programmer to program at the interface instead of the various implementations.

Interface Naming Convention: Use an adjective (typically ends with `"able"`) consisting of one or more words. Each word shall be initial capitalized (camel-case). For example, `Serializable`, `Externalizable`, `Movable`, `Clonable`, `Runnable`, **etc.**

We can re-write the `abstract` superclass `Shape` into an interface, containing only `abstract` methods, as follows:



```
/**
 * The interface Shape specifies the behaviors
 * of this implementations subclasses.
 */
public interface Shape { // Use keyword "interface" instead of
    "class"
    // List of public abstract methods to be implemented by its
    subclasses
    // All methods in interface are "public abstract".
```

```
        // "protected", "private" and "package" methods are NOT
allowed.
```

```
        double getArea();
```

```
    }
```

```
=====
=====
```

```
/**
```

```
 * The subclass Rectangle needs to implement all the abstract
methods in Shape
```

```
 */
```

```
public class Rectangle implements Shape {    // using keyword
"implements" instead of "extends"
```

```
    // Private member variables
```

```
    private int length, width;
```

```
    /** Constructs a Rectangle instance with the given length and
width */
```

```
    public Rectangle(int length, int width) {
```

```
        this.length = length;
```

```
        this.width = width;
```

```
    }
```

```
    /** Returns a self-descriptive string */
```

```
    @Override
```

```
    public String toString() {
```

```
        return "Rectangle[length=" + length + ",width=" + width +
"]";
```

```
    }
```

```
    // Need to implement all the abstract methods defined in the
interface
```

```
    /** Returns the area of this rectangle */
```

```
    @Override
```

```
    public double getArea() {
```

```
        return length * width;
```

```
    }
```

```
}
```

```
=====
=====
```

```
/**
```

```
 * The subclass Triangle need to implement all the abstract
methods in Shape
```

```
 */
```

```
public class Triangle implements Shape {
```

```

// Private member variables
private int base, height;

    /** Constructs a Triangle instance with the given base and
height */
    public Triangle(int base, int height) {
        this.base = base;
        this.height = height;
    }

    /** Returns a self-descriptive string */
    @Override
    public String toString() {
        return "Triangle[base=" + base + ",height=" + height + "];"
    }

    // Need to implement all the abstract methods defined in the
interface
    /** Returns the area of this triangle */
    @Override
    public double getArea() {
        return 0.5 * base * height;
    }
}

```

```

=====
==

```

A test driver is as follows:

```

public class TestShape {
    public static void main(String[] args) {
        Shape s1 = new Rectangle(1, 2); // upcast
        System.out.println(s1);
        //Rectangle[length=1,width=2]
        System.out.println("Area is " + s1.getArea());
        //Area is 2.0

        Shape s2 = new Triangle(3, 4); // upcast
        System.out.println(s2);
        //Triangle[base=3,height=4]
        System.out.println("Area is " + s2.getArea());
        //Area is 6.0

        // Cannot create instance of an interface
        //Shape s3 = new Shape("green");
    }
}

```

```
        //compilation error: Shape is abstract; cannot be
instantiated
    }

}
```